

DRC Results Generator

Moses Mburu

Table of contents

1 Purpose	3
2 Run Configuration	3
3 Build Descriptive Objects	7
4 Build Classical Decomposition Objects	9
5 Fit Concentration-Index Trees	22
6 Fit Lecturer Rpart Comparison Trees	28
6.0.1 Lecturer rpart method 1: absolute CI impurity	30
6.0.2 Lecturer rpart method 2: robust absolute CI impurity	30
7 Forest and SHAP Objects	31
8 Compare Methods	50
9 Save Results Objects	52

```
knitr::opts_chunk$set(  
  fig.width = 9,  
  fig.height = 5.5,  
  dpi = 500,  
  dev = "png"  
)  
  
library(data.table)  
library(ggplot2)  
library(grid)  
library(knitr)  
library(ineqTrees)  
library(partykit)  
library(here)  
source(here("R", "report_helpers.R"))  
source(here("R", "ci_rpart_comparison_methods.R"))  
  
kableExtra::use_latex_packages()
```

```

env_int <- function(name, default) {
  value <- suppressWarnings(as.integer(Sys.getenv(name, unset = NA_character_)))
  as.integer(ifelse(is.na(value) || value <= 0L, default, value))
}

env_num_or_null <- function(name, default = NULL) {
  value <- Sys.getenv(name, unset = NA_character_)
  if (is.na(value) || !nzchar(value)) {
    return(default)
  }

  parsed <- suppressWarnings(as.numeric(value))
  if (is.na(parsed)) {
    return(default)
  }
  parsed
}

```

```

default_params <- list(
  analysis_rows = 1L,
  tuning_folds = 10L,
  undersample = 0.41
)

runtime_params <- utils::modifyList(default_params, as.list(params))

runtime_params$analysis_rows <- env_int("ANALYSIS_ROWS", runtime_params$analysis_rows)
runtime_params$tuning_folds <- max(2L, env_int("TUNING_FOLDS", runtime_params$tuning_folds))
runtime_params$undersample <- env_num_or_null("UNDERSAMPLE", runtime_params$undersample)

shap_n <- 300L
shap_nsim <- 8L

```

```

criterion_types <- c("CI", "CIg", "CIc", "L")
rineq_ci_types <- c("CI", "CIg", "CIc", "CIw")
results_object_file <- here("data", "drc_report_results_objects.rda")
source_data_file <- here("prev_analysis", "DR Congo .dta")
paper_reference_file <- here("prev_analysis", "667.full.pdf")
outcome_labels <- c(
  alive = "Alive at age 5",
  died = "Died before age 5"
)

main_tree_control <- ci_tree_control(
  minsplit = 500L,
  minbucket = 250L,
  minprob = 0.02,
  maxdepth = 5L,
  min_gain = 0.00001,
  min_relative_gain = 0.20
)

```

```

map_model_terms_to_predictors <- function(term_names, predictors) {
  vapply(term_names, function(term) {
    hit <- predictors[term == predictors | startsWith(term, predictors)]
    c(hit[which.max(nchar(hit))], term)[1L]
  }, character(1L))
}

```

```
}
```

```
canonicalize_model_terms <- function(term_names, reference_terms) {  
  lookup <- data.table(  
    variable = reference_terms,  
    syntactic = make.names(reference_terms, unique = FALSE)  
  )  
  lookup <- lookup[  
    !duplicated(syntactic) & !duplicated(syntactic, fromLast = TRUE)  
  ]  
  
  out <- term_names  
  exact <- match(out, reference_terms)  
  out[!is.na(exact)] <- reference_terms[exact[!is.na(exact)]]  
  
  needs_lookup <- which(is.na(exact))  
  syntactic <- match(out[needs_lookup], lookup$syntactic)  
  out[needs_lookup[!is.na(syntactic)]] <- lookup$variable[syntactic[!is.na(syntactic)]]  
  
  out  
}
```

```
rank_from_named_vector <- function(x, method_name) {  
  x <- x %||% numeric()  
  data.table(variable = names(x), score = as.numeric(x))[  
    is.finite(score) & score != 0  
  ][  
    order(-abs(score))  
  ][  
    ,  
    .(variable, method = method_name, rank = seq_len(.N), score)  
  ]  
}
```

1 Purpose

This document is the canonical generator for the objects used by the main DRC results report. It is written as a readable workflow so a lecturer can follow what is loaded, what is fitted, what is compared, and what is finally saved.

If you need a plain R script for batch execution, extract it from this document with `knitr::purl("Generate_DRC_Results.R")`. Display-only chunks are excluded from the purl output so the extracted script stays focused on computation and saving.

2 Run Configuration

The runtime of this generator is controlled by two quantities: the number of analysis rows and the number of tuning folds. The remaining modelling choices are fixed below so the workflow stays readable. Set `analysis_rows` to 1 to use the full data set.

```
required_source_packages <- c("haven")  
missing_source_packages <- required_source_packages[  
  !vapply(required_source_packages, requireNamespace, logical(1L), quietly = TRUE)  
]
```

```

stopifnot(length(missing_source_packages) == 0L)

drc_2007_dt <- as.data.table(haven::read_dta(source_data_file))

factor_from_codes <- function(x, labels) {
  factor(labels[as.character(as.integer(x))], levels = unname(labels))
}

binary_factor <- function(x, labels) {
  factor(labels[as.character(as.integer(x))], levels = unname(labels))
}

drc_birth_labels <- c(
  "1" = "First birth",
  "2" = "2-4 short interval",
  "3" = "2-4 long interval",
  "4" = "5+ short interval",
  "5" = "5+ long interval"
)

drc_wealth_group_labels <- c("1" = "High household wealth", "2" = "Low household wealth")
drc_age_mother_labels <- c("1" = "20 or more", "2" = "Less than 20")
drc_education_labels <- c("1" = "Any education", "2" = "No education")
drc_occupation_labels <- c(
  "1" = "Other",
  "2" = "Household, unskilled, not working",
  "3" = "Agriculture"
)

drc_region_labels <- c(
  "1" = "Bandundu",
  "2" = "Bas-Congo",
  "3" = "Equateur",
  "4" = "Kasai Occidental",
  "5" = "Kasai Oriental",
  "6" = "Katanga",
  "7" = "Kinshasa",
  "8" = "Maniema",
  "9" = "Nord-Kivu",
  "10" = "Orientale",
  "11" = "Sud-Kivu"
)

raw_congo_model_dt <- drc_2007_dt[
  ,
  .(
    wealth = as.numeric(wealth),
    deadu5_num = as.integer(deadu5),
    sample_weight = as.numeric(weight),
    quint = factor_from_codes(quint, drc_wealth_group_labels),
    unskilled = binary_factor(
      unskilled,
      c("0" = "Skilled birth attendance", "1" = "No skilled birth attendance")
    ),
    male = binary_factor(male, c("0" = "Female", "1" = "Male")),
    birth = factor_from_codes(birth, drc_birth_labels),
    agemoth = factor_from_codes(agemoth, drc_age_mother_labels),
    rural = binary_factor(rural, c("0" = "Urban", "1" = "Rural")),
    ed = factor_from_codes(ed, drc_education_labels),
    ped = factor_from_codes(ped, drc_education_labels),
  )
]

```

```

    mocc = factor_from_codes(mocc, drc_occupation_labels),
    pocc = factor_from_codes(pocc, drc_occupation_labels),
    reg = factor_from_codes(reg, drc_region_labels)
  )
]
raw_congo_model_dt <- na.omit(raw_congo_model_dt)

raw_congo_var_labels <- c(
  quint = "Household wealth",
  unskilled = "Skilled birth attendance",
  male = "Sex of the child",
  birth = "Birth order and interval",
  agemoth = "Mother's age at birth",
  rural = "Type of residence",
  ed = "Mother's education",
  ped = "Father's education",
  mocc = "Mother's occupation",
  pocc = "Father's occupation",
  reg = "Region"
)
raw_congo_predictors <- names(raw_congo_var_labels)

```

```

raw_congo_model_dt <- sample_analysis_rows(
  raw_congo_model_dt,
  use_full = runtime_params$analysis_rows == 1L,
  n = runtime_params$analysis_rows,
  seed = 20260516
)

```

```

predictor_level_counts <- vapply(
  raw_congo_model_dt[, raw_congo_predictors, with = FALSE],
  function(x) length(levels(as.factor(x))),
  integer(1L)
)
nominal_predictors <- raw_congo_predictors[
  vapply(
    raw_congo_model_dt[, raw_congo_predictors, with = FALSE],
    function(x) is.factor(x) || is.character(x),
    logical(1L)
  )
]
multi_level_predictors <- intersect(
  names(predictor_level_counts[predictor_level_counts > 2L]),
  nominal_predictors
)
kept_predictors <- setdiff(raw_congo_predictors, multi_level_predictors)

make_indicator_name <- function(variable, level) {
  make.names(paste(variable, level, sep = "_"), unique = FALSE)
}

indicator_level_map <- rbindlist(lapply(multi_level_predictors, function(variable) {
  levels_i <- levels(as.factor(raw_congo_model_dt[[variable]]))
  data.table(
    raw_variable = variable,
    level = levels_i,
    variable = make_indicator_name(variable, levels_i),

```

```

      variable_label = paste0(unname(raw_congo_var_labels[variable]), ":", levels_i)
    )
  }, fill = TRUE)

stopifnot(!anyDuplicated(indicator_level_map$variable))

model_predictor_dt <- copy(raw_congo_model_dt[, kept_predictors, with = FALSE])
for (i in seq_len(nrow(indicator_level_map))) {
  source_variable <- indicator_level_map$raw_variable[i]
  source_level <- indicator_level_map$level[i]
  target_variable <- indicator_level_map$variable[i]
  model_predictor_dt[
    ,
    (target_variable) := factor(
      fifelse(as.character(raw_congo_model_dt[[source_variable]]) == source_level, "Yes", "No"),
      levels = c("No", "Yes")
    )
  ]
}

congo_model_dt <- data.table(
  raw_congo_model_dt[, .(wealth, deadu5_num, sample_weight)],
  model_predictor_dt
)

congo_predictors <- names(model_predictor_dt)
congo_var_labels <- setNames(congo_predictors, congo_predictors)
congo_var_labels[intersect(kept_predictors, names(raw_congo_var_labels))] <-
  raw_congo_var_labels[intersect(kept_predictors, names(raw_congo_var_labels))]
congo_var_labels[indicator_level_map$variable] <- indicator_level_map$variable_label

congo_ci_formula <- stats::as.formula(
  paste("cbind(wealth, deadu5_num) ~", paste(congo_predictors, collapse = " + "))
)

canonicalize_regression_terms <- function(term_names) {
  vapply(term_names, function(term) {
    parent <- map_model_terms_to_predictors(term, raw_congo_predictors)
    source_level <- sub(paste0("^", parent), "", term)
    hit <- indicator_level_map[raw_variable == parent & level == source_level]
    choices <- c(hit$variable[1L], parent, term)
    choice_id <- which(c(
      parent %in% multi_level_predictors && parent != term && nrow(hit) > 0L,
      parent %in% raw_congo_predictors && parent != term,
      TRUE
    ))[1L]
    choices[choice_id]
  }, character(1L))
}

format_variable_label <- function(x) {
  vapply(x, function(value) {
    parent <- map_model_terms_to_predictors(value, raw_congo_predictors)
    suffix <- trimws(sub(paste0("^", parent), "", value))
    suffix <- gsub("^[:. _]+", "", suffix)
    parent_label <- unname(raw_congo_var_labels[parent])
    choices <- c(
      NA_character_,
      unname(congo_var_labels[value]),

```

```

        unname(raw_congo_var_labels[value]),
        paste0(parent_label, ": ", suffix),
        parent_label,
        value
    )
    choice_id <- which(c(
        is.na(value) || !nzchar(value),
        value %in% names(congo_var_labels),
        value %in% names(raw_congo_var_labels),
        parent %in% names(raw_congo_var_labels) && parent != value && nzchar(suffix),
        parent %in% names(raw_congo_var_labels) && parent != value,
        TRUE
    ))[1L]
    choices[choice_id]
}, character(1L))
}

```

```

report_compact_table(
  data.table(
    item = c(
      "Source data",
      "Rows requested",
      "Rows used",
      "Tuning folds",
      "Published benchmark",
      "Candidate predictors",
      "Output file"
    ),
    value = c(
      source_data_file,
      fifelse(
        runtime_params$analysis_rows == 1L,
        "Full data set",
        format(runtime_params$analysis_rows, big.mark = ",", scientific = FALSE)
      ),
      format(nrow(congo_model_dt), big.mark = ",", scientific = FALSE),
      runtime_params$tuning_folds,
      "Van Malderen et al. 2013 Table 3, DR Congo 2007",
      length(congo_predictors),
      results_object_file
    )
  ),
  caption = "Run configuration for the DRC results generator",
  column_widths = c("4.5cm", "9cm")
)

```

Table 1: Run configuration for the DRC results generator

item	value
Source data	C:/Users/moses.mburu.FIND/Pictures/personal/thesis-mse/prev_analysis/DR Congo .dta
Rows requested	Full data set
Rows used	8,296
Tuning folds	10
Published benchmark	Van Malderen et al. 2013 Table 3, DR Congo 2007

3 Build Descriptive Objects

This section constructs the basic descriptive outputs that anchor the rest of the report: outcome prevalence, predictor coverage, and root-node inequality.

```
outcome_plot_dt <- congo_model_dt[
  ,
  .(weighted_n = sum(sample_weight)),
  by = .(deadu5_num)
][
  ,
  outcome_label := fifelse(deadu5_num == 1, outcome_labels[["died"]], outcome_labels[["alive"]])
][
  ,
  weighted_percent := 100 * weighted_n / sum(weighted_n)
]

candidate_predictor_table <- rbindlist(lapply(raw_congo_predictors, function(var) {
  x <- raw_congo_model_dt[[var]]
  level_dt <- raw_congo_model_dt[, .(weighted_n = sum(sample_weight)), by = var][order(-weighted_n)]
  level_dt[, weighted_percent := 100 * weighted_n / sum(weighted_n)]
  data.table(
    variable = var,
    label = unname(raw_congo_var_labels[var]),
    class = paste(class(x), collapse = "/"),
    n_levels = uniqueN(x),
    most_common_level = as.character(level_dt[[var]][1L]),
    most_common_percent = level_dt$weighted_percent[1L]
  )
}), fill = TRUE)

drc_root_ci_table <- rbindlist(lapply(criterion_types, function(type) {
  ci_fun <- ci_factory(type)
  data.table(
    criterion = type,
    estimate = ci_fun(
      cbind(congo_model_dt$wealth, congo_model_dt$deadu5_num),
      congo_model_dt$sample_weight
    )
  )
}))

drc_data_summary_table <- data.table(
  item = c(
    "Analysis rows",
    "Weighted sample size",
    "Weighted under-five mortality",
    "Ranking variable",
    "Candidate predictors"
  ),
  value = c(
    format(nrow(congo_model_dt), big.mark = ",", scientific = FALSE),
```



```

format(round(sum(congo_model_dt$sample_weight)), big.mark = ",", scientific = FALSE),
sprintf("%.2f%%", 100 * weighted_mean_safe(congo_model_dt$deadu5_num, congo_model_dt$sample_w
"DHS wealth index",
paste(unname(raw_congo_var_labels), collapse = "; ")
)
)

```

```

report_compact_table(
  drc_data_summary_table,
  caption = "Analysis sample summary used by the DRC results report",
  column_widths = c("4cm", "10cm")
)

```

Table 2: Analysis sample summary used by the DRC results report

item	value
Analysis rows	8,296
Weighted sample size	8,323
Weighted under-five mortality	10.32%
Ranking variable	DHS wealth index
Candidate predictors	Household wealth; Skilled birth attendance; Sex of the child; Birth order and interval; Mother's age at birth; Type of residence; Mother's education; Father's education; Mother's occupation; Father's occupation; Region

```

report_compact_table(
  drc_root_ci_table,
  digits = 4,
  caption = "Root-node concentration-index estimates across criteria"
)

```

Table 3: Root-node concentration-index estimates across criteria

criterion	estimate
CI	0.1156
CIg	0.0119
CIc	0.0477
L	0.1935

4 Build Classical Decomposition Objects

This section now follows a result-first workflow. The survey model is fitted in its own chunk, the summary is shown immediately after, and each `rineq` decomposition method is built in its own chunk before being displayed.

```

library(survey)
library(rineq)
linear_decomp_formula <- stats::as.formula(
  paste("deadu5_num ~", paste(raw_congo_predictors, collapse = " + "))
)

```

```
linear_decomp_design <- survey::svydesign(
  ids = ~1,
  weights = ~sample_weight,
  data = raw_congo_model_dt
)
linear_decomp_fit <- survey::svyglm(
  linear_decomp_formula,
  design = linear_decomp_design,
  family = stats::quasibinomial()
)
```

```
print(summary(linear_decomp_fit))
```

Call:

```
svyglm(formula = linear_decomp_formula, design = linear_decomp_design,
  family = stats::quasibinomial())
```

Survey design:

```
survey::svydesign(ids = ~1, weights = ~sample_weight, data = raw_congo_model_dt)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-2.908431	0.275345	-10.563	<2e-16 ***
quintLow household wealth	0.068117	0.125237	0.544	0.5865
unskilledNo skilled birth attendance	0.198131	0.111115	1.783	0.0746 .
maleMale	0.241990	0.105496	2.294	0.0218 *
birth2-4 short interval	-0.003766	0.185540	-0.020	0.9838
birth2-4 long interval	-0.509098	0.168982	-3.013	0.0026 **
birth5+ short interval	0.419653	0.210396	1.995	0.0461 *
birth5+ long interval	-0.396393	0.210666	-1.882	0.0599 .
agemothLess than 20	0.032723	0.142687	0.229	0.8186
ruralRural	0.090066	0.154181	0.584	0.5591
edNo education	0.336221	0.131606	2.555	0.0106 *
pedNo education	-0.035664	0.127535	-0.280	0.7798
moccHousehold, unskilled, not working	-0.017693	0.155378	-0.114	0.9093
moccAgriculture	0.140468	0.163233	0.861	0.3895
poccHousehold, unskilled, not working	0.235475	0.162445	1.450	0.1472
poccAgriculture	0.224936	0.149083	1.509	0.1314
regBas-Congo	0.258066	0.243006	1.062	0.2883
regEquateur	0.249695	0.237621	1.051	0.2934
regKasai Occidental	0.105100	0.228244	0.460	0.6452
regKasai Oriental	0.152682	0.240257	0.635	0.5251
regKatanga	0.515018	0.228125	2.258	0.0240 *
regKinshasa	0.081422	0.262411	0.310	0.7564
regManiema	0.488879	0.224476	2.178	0.0294 *
regNord-Kivu	-0.251522	0.293512	-0.857	0.3915
regOrientale	-0.083627	0.264827	-0.316	0.7522
regSud-Kivu	0.299991	0.270055	1.111	0.2667

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for quasibinomial family taken to be 1.013466)

Number of Fisher Scoring iterations: 5

```

linear_x <- stats::model.matrix(linear_decomp_formula, data = raw_congo_model_dt)
linear_beta <- stats::coef(linear_decomp_fit)
linear_terms <- setdiff(colnames(linear_x), "(Intercept)")

linear_term_info <- data.table(
  term = linear_terms,
  variable = canonicalize_regression_terms(linear_terms),
  linear_coefficient = unname(linear_beta[linear_terms]),
  mean_of_variable = vapply(
    linear_terms,
    function(term) weighted_mean_safe(linear_x[, term], raw_congo_model_dt$sample_weight),
    numeric(1L)
  )
)

regression_model_summary <- as.data.table(
  as.data.frame(broom::tidy(linear_decomp_fit, conf.int = TRUE))
)
regression_model_summary[, variable := fifelse(
  term == "(Intercept)",
  term,
  canonicalize_regression_terms(term)
)]
regression_model_summary[, term_label := fifelse(
  term == "(Intercept)",
  "Intercept",
  format_variable_label(variable)
)]
data.table::setcolorder(
  regression_model_summary,
  c("term", "term_label", "variable", setdiff(names(regression_model_summary), c("term", "term_label")))
)

decomposition_to_table <- function(object, digits = 4L) {
  data.table(
    term = names(object$rel_contribution),
    `Contribution (%)` = round(object$rel_contribution, digits),
    `Contribution (Abs)` = round(object$ci_contribution, digits),
    Elasticity = round(c(0, object$elasticities), digits),
    `Concentration Index` = c(NA_real_, round(object$partial_cis, digits)),
    `lower 5%` = c(NA_real_, round(object$confints[1, ], digits)),
    `upper 5%` = c(NA_real_, round(object$confints[2, ], digits)),
    Corrected = c("", ifelse(object$corrected_coefficients, "yes", "no"))
  )
}

summarise_rineq_decomposition <- function(object, method_name) {
  out <- merge(
    linear_term_info,
    decomposition_to_table(object, digits = 6L)[
      term != "residual",
      .(
        term,
        elasticity = Elasticity,
        concentration_index_of_variable = `Concentration Index`,
        regression_contribution = `Contribution (Abs)`
      )
    ],

```

```

    by = "term",
    all.y = TRUE
  )
  out[, variable := fifelse(!is.na(variable) & nzchar(variable), variable, term)]
  out[, `:=`(
    criterion = method_name,
    variable_label = format_variable_label(variable)
  )]

total_abs_contribution <- out[, sum(abs(regression_contribution), na.rm = TRUE)]
out[, regression_pct_contribution := fifelse(
  rep(is.finite(total_abs_contribution) && total_abs_contribution > .Machine$double.eps, .N),
  100 * abs(regression_contribution) / total_abs_contribution,
  NA_real_
)]

data.table::setcolorder(
  out,
  c(
    "variable",
    "variable_label",
    "criterion",
    "linear_coefficient",
    "mean_of_variable",
    "elasticity",
    "concentration_index_of_variable",
    "regression_contribution",
    "regression_pct_contribution"
  )
)
out[order(-abs(regression_pct_contribution))]
}

```

```

rineq_ci_decomp <- rineq::decomposition(
  outcome = raw_congo_model_dt$deadu5_num,
  betas = unname(linear_beta[linear_terms]),
  mm = linear_x[, linear_terms, drop = FALSE],
  ranker = raw_congo_model_dt$wealth,
  wt = raw_congo_model_dt$sample_weight,
  correction = TRUE,
  citype = "CI"
)
rineq_ci_summary <- decomposition_to_table(rineq_ci_decomp, digits = 4L)
classical_ci_table <- summarise_rineq_decomposition(rineq_ci_decomp, "CI")

```

```

report_compact_table(
  rineq_ci_summary,
  digits = 4,
  caption = "rineq decomposition summary: CI"
)

```

Table 4: rineq decomposition summary: CI

	term	Contribution (%)	Contribution (Abs)	Elasticity	Concen
	residual	-975.5357	1.1266	0.0000	
	quintLow household wealth	140.6699	-0.1624	0.2885	

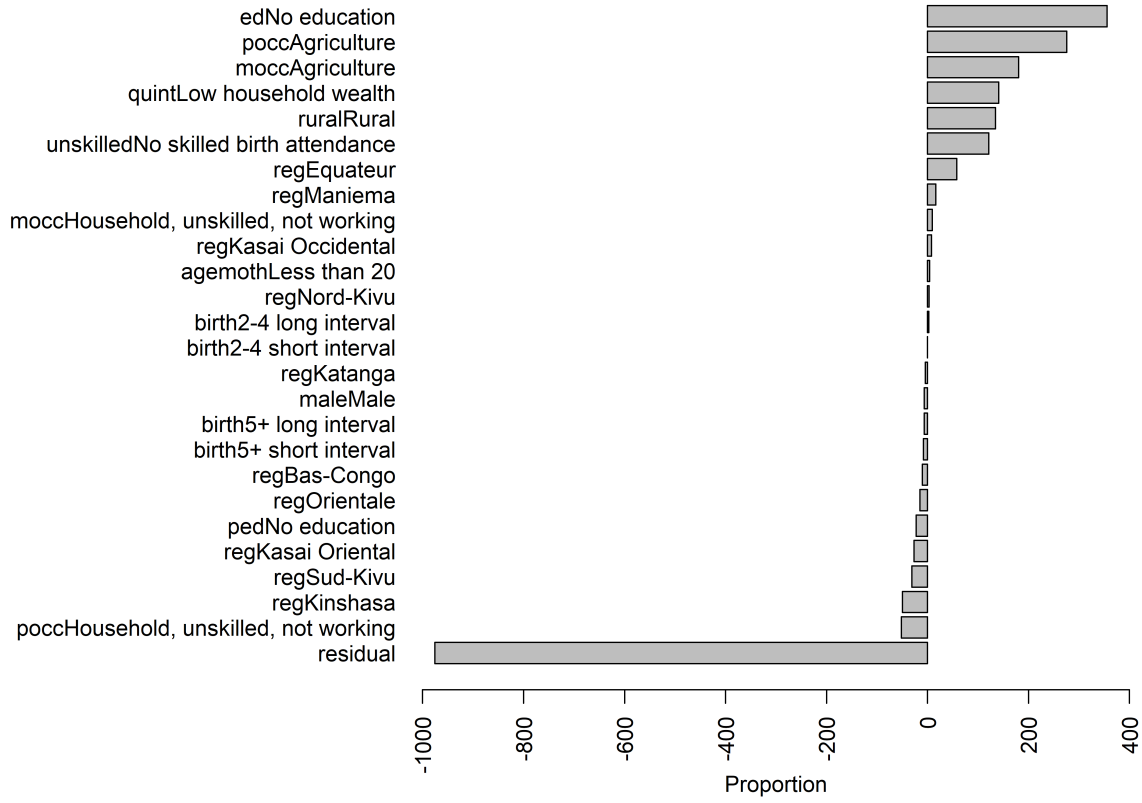
unskilledNo skilled birth attendance	121.0994	-0.1398	1.1080
maleMale	-6.4425	0.0074	1.1564
birth2-4 short interval	-0.0587	0.0001	-0.0039
birth2-4 long interval	2.3826	-0.0028	-1.6719
birth5+ short interval	-8.1400	0.0094	0.4140
birth5+ long interval	-6.7469	0.0078	-1.0078
agemothLess than 20	4.0288	-0.0047	0.1370
ruralRural	134.7545	-0.1556	0.5377
edNo education	355.7900	-0.4109	1.8406
pedNo education	-22.5539	0.0260	-0.0932
moccHousehold, unskilled, not working	9.2447	-0.0107	-0.0397
moccAgriculture	180.2946	-0.2082	0.7603
poccHousehold, unskilled, not working	-52.0012	0.0601	0.2933
poccAgriculture	275.6313	-0.3183	1.1003
regBas-Congo	-10.4632	0.0121	0.1003
regEquateur	58.0391	-0.0670	0.3108
regKasai Occidental	7.5561	-0.0087	0.1083
regKasai Oriental	-26.9466	0.0311	0.1970
regKatanga	-4.5699	0.0053	0.5145
regKinshasa	-49.7000	0.0574	0.0719
regManiema	16.5843	-0.0192	0.1668
regNord-Kivu	3.2146	-0.0037	-0.0949
regOrientale	-15.1820	0.0175	-0.0936
regSud-Kivu	-30.9490	0.0357	0.1400

```

plot(
  rineq_ci_decomp,
  horiz = TRUE,
  decreasing = FALSE,
  main = "rineq decomposition contributions: CI"
)

```

rineq decomposition contributions: CI



```
rineq_cig_decomp <- rineq::decomposition(
  outcome = raw_congo_model_dt$deadu5_num,
  betas = unname(linear_beta[linear_terms]),
  mm = linear_x[, linear_terms, drop = FALSE],
  ranker = raw_congo_model_dt$wealth,
  wt = raw_congo_model_dt$sample_weight,
  correction = TRUE,
  citype = "CIg"
)
rineq_cig_summary <- decomposition_to_table(rineq_cig_decomp, digits = 4L)
classical_cig_table <- summarise_rineq_decomposition(rineq_cig_decomp, "CIg")
```

```
report_compact_table(
  rineq_cig_summary,
  digits = 4,
  caption = "rineq decomposition summary: CIg"
)
```

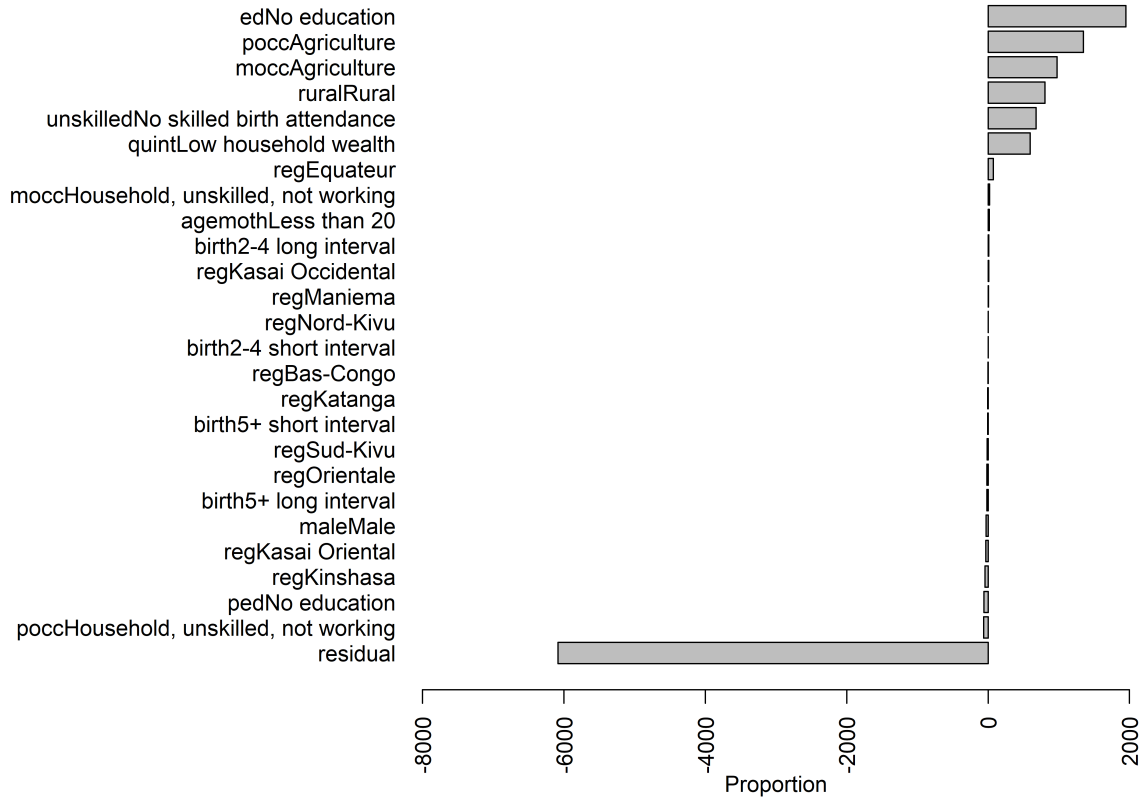
Table 5: rineq decomposition summary: CIg

	term	Contribution (%)	Contribution (Abs)	Elasticity	Concen
	residual	-6083.3897	0.7249	0.0000	
	quintLow household wealth	595.8302	-0.0710	0.2885	
	unskilledNo skilled birth attendance	677.2228	-0.0807	1.1080	
	maleMale	-30.7864	0.0037	1.1564	

birth2-4 short interval	-0.0605	0.0000	-0.0039
birth2-4 long interval	7.8246	-0.0009	-1.6719
birth5+ short interval	-8.0306	0.0010	0.4140
birth5+ long interval	-17.1537	0.0020	-1.0078
agemothLess than 20	16.8616	-0.0020	0.1370
ruralRural	804.5451	-0.0959	0.5377
edNo education	1947.7168	-0.2321	1.8406
pedNo education	-58.9303	0.0070	-0.0932
moccHousehold, unskilled, not working	20.7690	-0.0025	-0.0397
moccAgriculture	975.8716	-0.1163	0.7603
poccHousehold, unskilled, not working	-64.7787	0.0077	0.2933
poccAgriculture	1348.2802	-0.1607	1.1003
regBas-Congo	-4.0657	0.0005	0.1003
regEquateur	72.2531	-0.0086	0.3108
regKasai Occidental	7.7831	-0.0009	0.1083
regKasai Oriental	-34.7631	0.0041	0.1970
regKatanga	-4.5651	0.0005	0.5145
regKinshasa	-43.8701	0.0052	0.0719
regManiema	5.6569	-0.0007	0.1668
regNord-Kivu	1.2135	-0.0001	-0.0949
regOrientale	-16.9917	0.0020	-0.0936
regSud-Kivu	-14.4431	0.0017	0.1400

```
plot(
  rineq_cig_decomp,
  horiz = TRUE,
  decreasing = FALSE,
  main = "rineq decomposition contributions: CIG"
)
```

rineq decomposition contributions: CIc



```
rineq_cic_decomp <- rineq::decomposition(
  outcome = raw_congo_model_dt$deadu5_num,
  betas = unname(linear_beta[linear_terms]),
  mm = linear_x[, linear_terms, drop = FALSE],
  ranker = raw_congo_model_dt$wealth,
  wt = raw_congo_model_dt$sample_weight,
  correction = TRUE,
  citype = "CIc"
)
rineq_cic_summary <- decomposition_to_table(rineq_cic_decomp, digits = 4L)
classical_cic_table <- summarise_rineq_decomposition(rineq_cic_decomp, "CIc")
```

```
report_compact_table(
  rineq_cic_summary,
  digits = 4,
  caption = "rineq decomposition summary: CIc"
)
```

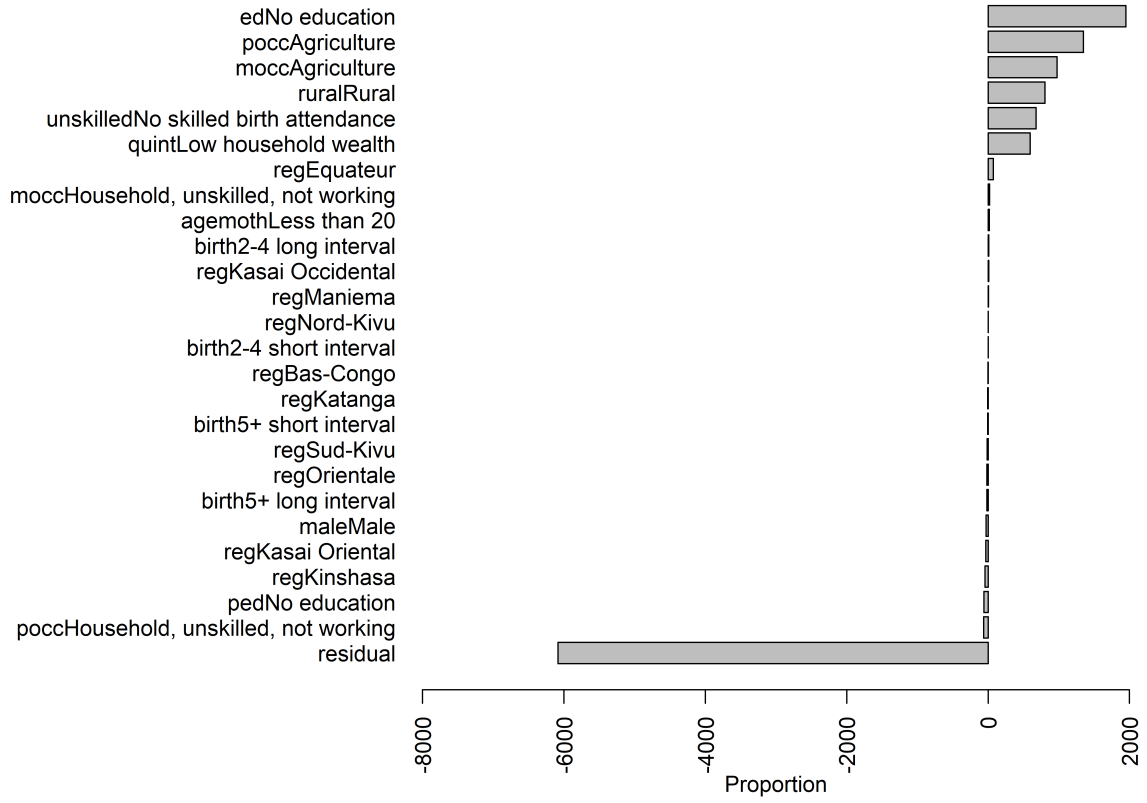
Table 6: rineq decomposition summary: CIc

	term	Contribution (%)	Contribution (Abs)	Elasticity	Concen
	residual	-6083.3897	2.8995	0.0000	
	quintLow household wealth	595.8302	-0.2840	0.2885	
	unskilledNo skilled birth attendance	677.2228	-0.3228	1.1080	
	maleMale	-30.7864	0.0147	1.1564	

birth2-4 short interval	-0.0605	0.0000	-0.0039
birth2-4 long interval	7.8246	-0.0037	-1.6719
birth5+ short interval	-8.0306	0.0038	0.4140
birth5+ long interval	-17.1537	0.0082	-1.0078
agemothLess than 20	16.8616	-0.0080	0.1370
ruralRural	804.5451	-0.3835	0.5377
edNo education	1947.7168	-0.9283	1.8406
pedNo education	-58.9303	0.0281	-0.0932
moccHousehold, unskilled, not working	20.7690	-0.0099	-0.0397
moccAgriculture	975.8716	-0.4651	0.7603
poccHousehold, unskilled, not working	-64.7787	0.0309	0.2933
poccAgriculture	1348.2802	-0.6426	1.1003
regBas-Congo	-4.0657	0.0019	0.1003
regEquateur	72.2531	-0.0344	0.3108
regKasai Occidental	7.7831	-0.0037	0.1083
regKasai Oriental	-34.7631	0.0166	0.1970
regKatanga	-4.5651	0.0022	0.5145
regKinshasa	-43.8701	0.0209	0.0719
regManiema	5.6569	-0.0027	0.1668
regNord-Kivu	1.2135	-0.0006	-0.0949
regOrientale	-16.9917	0.0081	-0.0936
regSud-Kivu	-14.4431	0.0069	0.1400

```
plot(
  rineq_cic_decomp,
  horiz = TRUE,
  decreasing = FALSE,
  main = "rineq decomposition contributions: CIc"
)
```

rineq decomposition contributions: CIc



```
rineq_ciw_decomp <- rineq::decomposition(
  outcome = raw_congo_model_dt$deadu5_num,
  betas = unname(linear_beta[linear_terms]),
  mm = linear_x[, linear_terms, drop = FALSE],
  ranker = raw_congo_model_dt$wealth,
  wt = raw_congo_model_dt$sample_weight,
  correction = TRUE,
  citype = "CIw"
)
rineq_ciw_summary <- decomposition_to_table(rineq_ciw_decomp, digits = 4L)
classical_ciw_table <- summarise_rineq_decomposition(rineq_ciw_decomp, "CIw")
```

```
report_compact_table(
  rineq_ciw_summary,
  digits = 4,
  caption = "rineq decomposition summary: CIw"
)
```

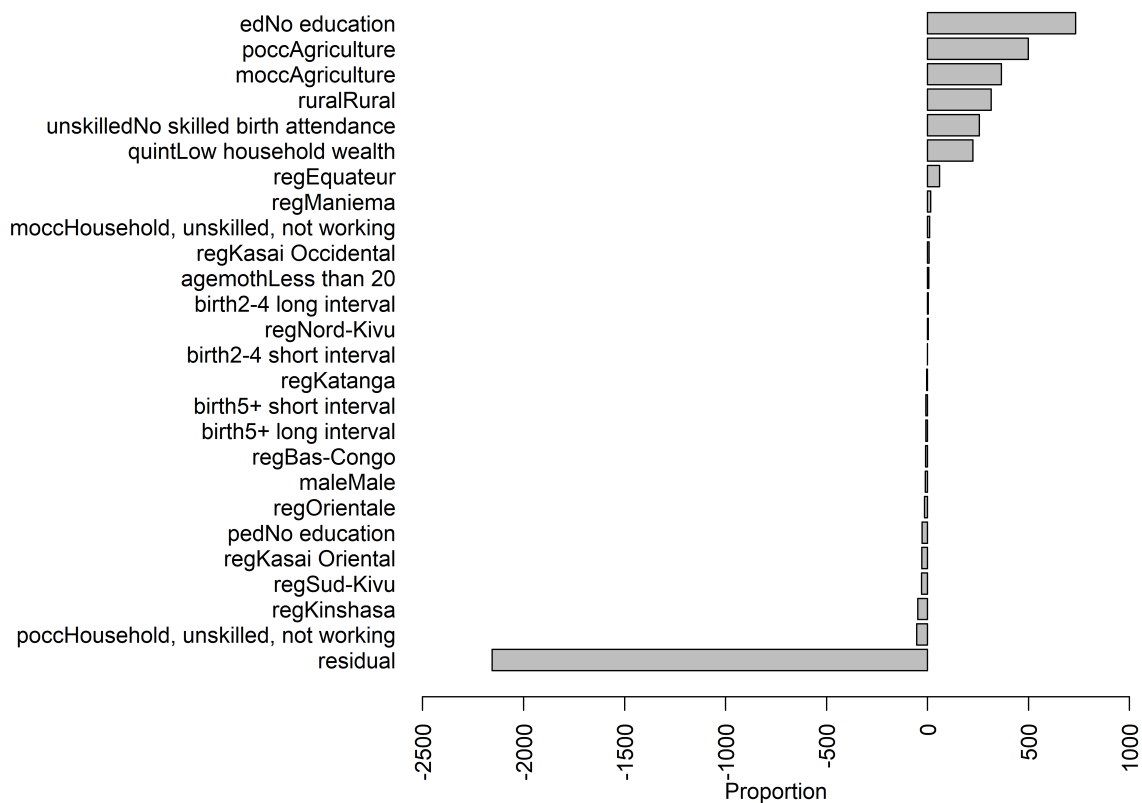
Table 7: rineq decomposition summary: CIw

	term	Contribution (%)	Contribution (Abs)	Elasticity	Concen
	residual	-2155.4990	2.7756	0.0000	
	quintLow household wealth	224.0948	-0.2886	0.2885	
	unskilledNo skilled birth attendance	256.7624	-0.3306	1.1080	
	maleMale	-11.3976	0.0147	1.1564	

birth2-4 short interval	-0.0589	0.0001	-0.0039
birth2-4 long interval	3.2319	-0.0042	-1.6719
birth5+ short interval	-8.1275	0.0105	0.4140
birth5+ long interval	-8.2025	0.0106	-1.0078
agemothLess than 20	6.3594	-0.0082	0.1370
ruralRural	314.7506	-0.4053	0.5377
edNo education	733.2702	-0.9442	1.8406
pedNo education	-27.6927	0.0357	-0.0932
moccHousehold, unskilled, not working	10.7926	-0.0139	-0.0397
moccAgriculture	366.2231	-0.4716	0.7603
poccHousehold, unskilled, not working	-53.5141	0.0689	0.2933
poccAgriculture	499.1007	-0.6427	1.1003
regBas-Congo	-9.7755	0.0126	0.1003
regEquateur	59.7219	-0.0769	0.3108
regKasai Occidental	7.5823	-0.0098	0.1083
regKasai Oriental	-27.8770	0.0359	0.1970
regKatanga	-4.5694	0.0059	0.5145
regKinshasa	-49.0382	0.0631	0.0719
regManiema	15.4156	-0.0199	0.1668
regNord-Kivu	2.9997	-0.0039	-0.0949
regOrientale	-15.3931	0.0198	-0.0936
regSud-Kivu	-29.1597	0.0375	0.1400

```
plot(
  rineq_ciw_decomp,
  horiz = TRUE,
  decreasing = FALSE,
  main = "rineq decomposition contributions: CIw"
)
```

rineq decomposition contributions: Clw



```
rineq_decomp_by_type <- list(
  CI = rineq_ci_decomp,
  CIg = rineq_cig_decomp,
  CIc = rineq_cic_decomp,
  Clw = rineq_ciw_decomp
)

rineq_summary_by_type <- list(
  CI = rineq_ci_summary,
  CIg = rineq_cig_summary,
  CIc = rineq_cic_summary,
  Clw = rineq_ciw_summary
)

rineq_summary <- rbindlist(lapply(names(rineq_summary_by_type), function(type) {
  out <- copy(rineq_summary_by_type[[type]])
  out[, criterion := type]
  data.table::setcolorder(out, c("criterion", setdiff(names(out), "criterion")))
  out
}), fill = TRUE)

classical_decomposition_table <- rbindlist(
  list(classical_ci_table, classical_cig_table, classical_cic_table, classical_ciw_table),
  fill = TRUE
)
```

```
published_drc_table3_reference <- rbindlist(list(
  data.table(
    criterion = "G",
    determinant = c(
      "Household wealth",
      "Skilled birth attendance",
      "Sex of the child",
      "Birth order and interval",
      "Mother's age at birth",
      "Type of residence",
      "Mother's education",
      "Father's education",
      "Mother's occupation",
      "Father's occupation",
      "Region"
    ),
    published_index = -0.128,
    published_percent = c(2.4, 7.3, 5.3, 35.6, 0.4, 3.4, 17.4, -0.7, 5.8, 9.1, 14.0)
  ),
  data.table(
    criterion = "C",
    determinant = c(
      "Household wealth",
      "Skilled birth attendance",
      "Sex of the child",
      "Birth order and interval",
      "Mother's age at birth",
      "Type of residence",
      "Mother's education",
      "Father's education",
      "Mother's occupation",
      "Father's occupation",
      "Region"
    ),
    published_index = 0.057,
    published_percent = c(13.1, 11.3, -0.6, -1.2, 0.4, 12.5, 33.1, -2.1, 17.6, 20.8, -4.9)
  )
))

determinant_map <- data.table(
  variable = c("quint", "unskilled", "male", "birth", "agemoth", "rural", "ed", "ped", "mocc", "pocc"),
  raw_variable = c("quint", "unskilled", "male", "birth", "agemoth", "rural", "ed", "ped", "mocc", "pocc"),
  determinant = c(
    "Household wealth",
    "Skilled birth attendance",
    "Sex of the child",
    "Birth order and interval",
    "Mother's age at birth",
    "Type of residence",
    "Mother's education",
    "Father's education",
    "Mother's occupation",
    "Father's occupation",
    "Region"
  )
)

determinant_map <- rbindlist(
  list(
```

```

    determinant_map,
    merge(
      indicator_level_map[, .(variable, raw_variable)],
      determinant_map[, .(raw_variable, determinant)],
      by = "raw_variable",
      all.x = TRUE,
      sort = FALSE
    )[, .(variable, raw_variable, determinant)]
  ),
  fill = TRUE
)

classical_decomposition_grouped <- classical_decomposition_table[
  determinant_map,
  on = "variable",
  nomatch = OL
][
  ,
  .(
    regression_contribution = sum(regression_contribution, na.rm = TRUE),
    regression_pct_contribution = sum(regression_pct_contribution, na.rm = TRUE)
  ),
  by = .(criterion, determinant)
]

published_drc_table3_comparison <- merge(
  published_drc_table3_reference[criterion == "C"],
  classical_decomposition_grouped[criterion == "CI"],
  by = "determinant",
  all.x = TRUE,
  sort = FALSE
)

published_drc_table3_comparison[
  ,
  difference_from_published := regression_pct_contribution - published_percent
]

```

```

report_compact_table(
  published_drc_table3_comparison[
    ,
    .(
      determinant,
      paper_c_percent = published_percent,
      generated_ci_percent = regression_pct_contribution,
      difference_from_published
    )
  ],
  digits = 2,
  caption = "Published DR Congo 2007 concentration-index decomposition compared with generated regression decomposition"
)

```

Table 8: Published DR Congo 2007 concentration-index decomposition compared with generated regression decomposition

determinant	paper_c_percent	generated_ci_percent	difference_from_published
Household wealth	13.1	9.12	-3.98

Skilled birth attendance	11.3	7.85	-3.45
Sex of the child	-0.6	0.42	1.02
Birth order and interval	-1.2	1.12	2.32
Mother's age at birth	0.4	0.26	-0.14
Type of residence	12.5	8.73	-3.77
Mother's education	33.1	23.06	-10.04
Father's education	-2.1	1.46	3.56
Mother's occupation	17.6	12.28	-5.32
Father's occupation	20.8	21.23	0.43
Region	-4.9	14.47	19.37

5 Fit Concentration-Index Trees

This is the core generator step. The grid is cross-validated, then each criterion is refitted at its selected setting.

```
tree_grid <- ci_tree_control_grid(
  minsplit = c(500L), # 750L worked
  minbucket = c(100L), # 100 worked
  minprob = 0.01,
  maxdepth = 10L,
  min_gain = 0.0001,
  min_relative_gain = c(0.15, 0.20, 0.25, 0.30)
)

# filter where minbucket is at least 50% of minsplit to avoid invalid settings
tree_grid <- tree_grid[minbucket <= 0.5 * minsplit]
```

```
ci_tuning_metrics <- c(
  "validation_gain",
  "relative_validation_gain",
  "percent_validation_root_recovered",
  "brier",
  "log_loss",
  "roc_auc"
)

tree_tuning <- tune_ci_tree(
  formula = congo_ci_formula,
  data = congo_model_dt,
  rank_name = "wealth",
  outcome_name = "deadu5_num",
  weights = congo_model_dt$sample_weight,
  type = criterion_types,
  control_grid = tree_grid,
  v = runtime_params$tuning_folds,
  strata = "deadu5_num",
  seed = 20260508,
  metric = ci_tuning_metrics,
  refit = FALSE,
  control = control_ci_tune(save_pred = TRUE)
)
```

```

tree_best_by_type <- ci_select_best(tree_tuning,
  metric = "relative_validation_gain"
)
tree_selection_table <- ci_fit_summary_table(
  tree_tuning,
  selected = tree_best_by_type,
  metrics = unique(tree_tuning$summary$metric),
  include_percent = FALSE
)
tree_selection_table[
  tree_best_by_type,
  terminal_nodes := i.mean_terminal_nodes,
  on = c("type", "grid_id")
]
tree_selection_table[, selection_basis := paste(
  runtime_params$tuning_folds,
  "fold cross-validation"
)]
report_compact_table(
  tree_selection_table,
  digits = 4,
  caption = "Summary of the concentration-index tree selection results across criteria"
)

```

type	grid_id	minsplit	minbucket	minprob	maxdepth	min_gain	min_relative_gain	n
CI	3	500	100	0.01	10	1e-04	0.25	
CIc	12	500	100	0.01	10	1e-04	0.30	
CIg	8	500	100	0.01	10	1e-04	0.30	
L	16	500	100	0.01	10	1e-04	0.30	

```

tree_models_by_type <- setNames(
  lapply(seq_len(nrow(tree_best_by_type)), function(i) {
    setting <- tree_best_by_type[i]
    ci_tree(
      formula = congo_ci_formula,
      data = congo_model_dt,
      rank_name = "wealth",
      outcome_name = "deadu5_num",
      weights = congo_model_dt$sample_weight,
      type = setting$type[1L],
      control = ci_control_from_row(setting)
    )
  }),
  tree_best_by_type$type
)

```

```

selected_tree_type <- tree_selection_table[order(-mean_validation_relative_gain)][1L, type]
selected_tree_fit <- tree_models_by_type[[selected_tree_type]]
selected_tree_summary <- as.data.table(ci_tree_terminal_summary(selected_tree_fit))

all_tree_terminal_summaries <- rbindlist(lapply(names(tree_models_by_type), function(type) {
  out <- as.data.table(ci_tree_terminal_summary(tree_models_by_type[[type]]))
  out[, criterion := type]
}))

```



```

    data.table::setcolorder(out, "criterion")
    out
  }), fill = TRUE)

tree_variable_importance <- collect_variable_importance(tree_models_by_type)

tree_best_controls <- tree_best_by_type[
  ,
  .(type, minsplit, minbucket, minprob, maxdepth, min_gain, min_relative_gain)
]

tree_complexity_metrics <- collect_complexity_metrics(tree_tuning)
tree_min_relative_gain_path <- min_relative_gain_path(tree_complexity_metrics,
  selected = tree_best_by_type
)

tree_fit_wide <- ci_fit_summary_table(
  tree_tuning,
  selected = tree_best_by_type,
  metrics = unique(tree_tuning$summary$metric),
  include_percent = FALSE
)
tree_fit_table <- ci_collect_metrics(
  tree_tuning,
  selected = tree_best_by_type,
  metric = unique(tree_tuning$summary$metric),
  format = "tidy",
  include_train = TRUE
)

report_compact_table(
  tree_selection_table,
  digits = 4,
  caption = "Selection summary for the fitted concentration-index trees"
)

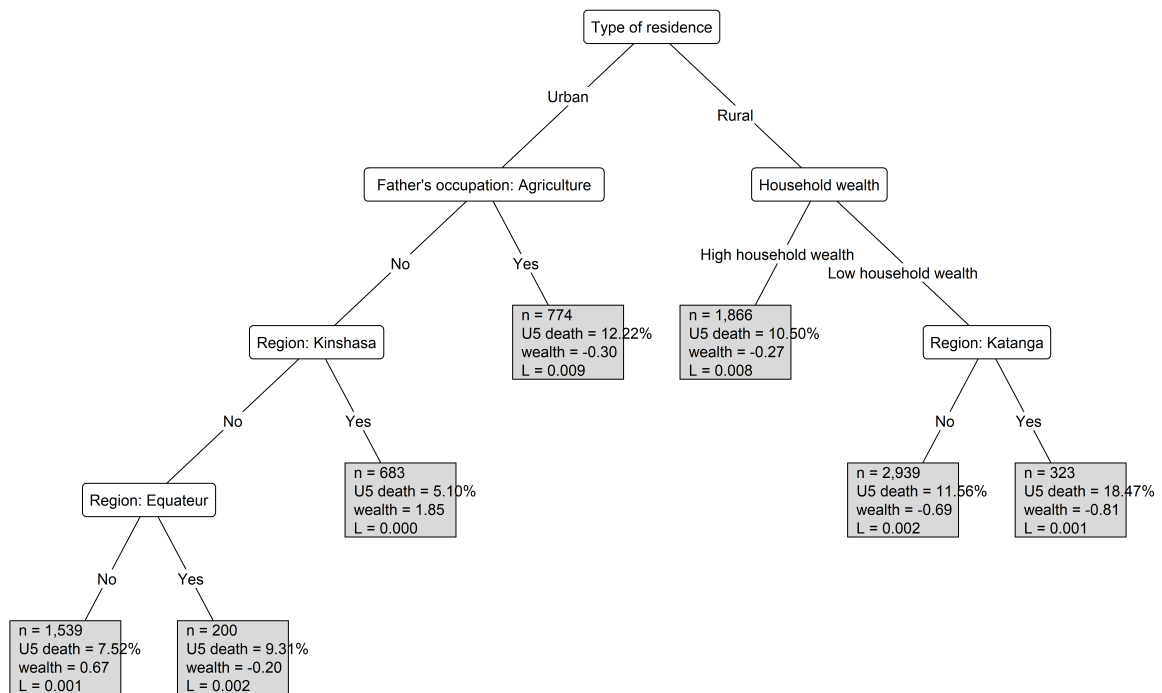
```

type	grid_id	minsplit	minbucket	minprob	maxdepth	min_gain	min_relative_gain	n
CI	3	500	100	0.01	10	1e-04	0.25	
CIc	12	500	100	0.01	10	1e-04	0.30	
CIg	8	500	100	0.01	10	1e-04	0.30	
L	16	500	100	0.01	10	1e-04	0.30	

```

ci_report_tree_plot(
  fit = selected_tree_fit,
  data = congo_model_dt,
  outcome_name = "deadu5_num",
  outcome_label = "U5 death",
  ci_type = "L",
  var_labels = congo_var_labels
)

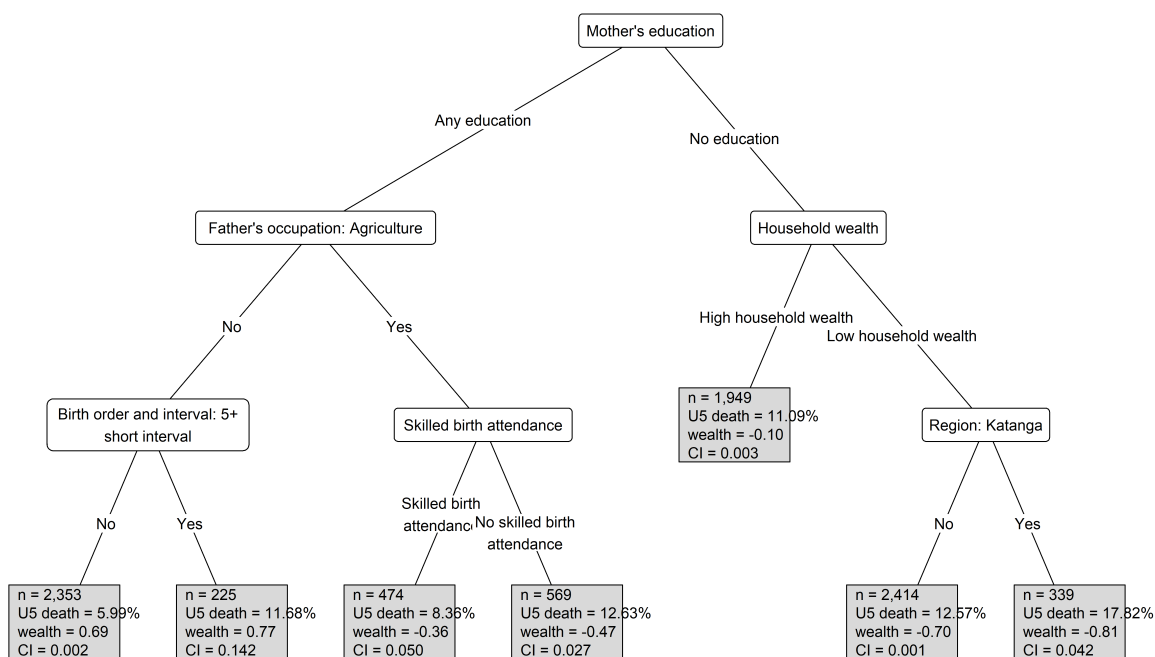
```



```

ci_report_tree_plot(
  fit = tree_models_by_type[["CI"]],
  data = congo_model_dt,
  outcome_name = "deadu5_num",
  outcome_label = "U5 death",
  ci_type = "CI",
  var_labels = congo_var_labels
)

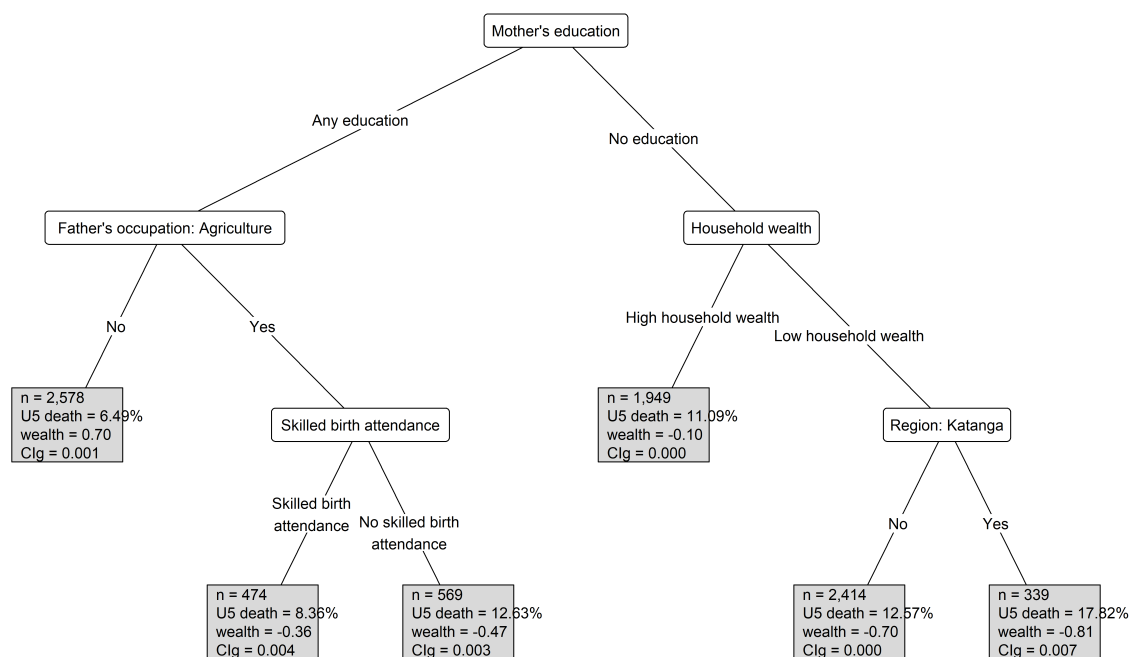
```



```

ci_report_tree_plot(
  fit = tree_models_by_type[["CIg"]],
  data = congo_model_dt,
  outcome_name = "deadu5_num",
  outcome_label = "U5 death",
  ci_type = "CIg",
  var_labels = congo_var_labels
)

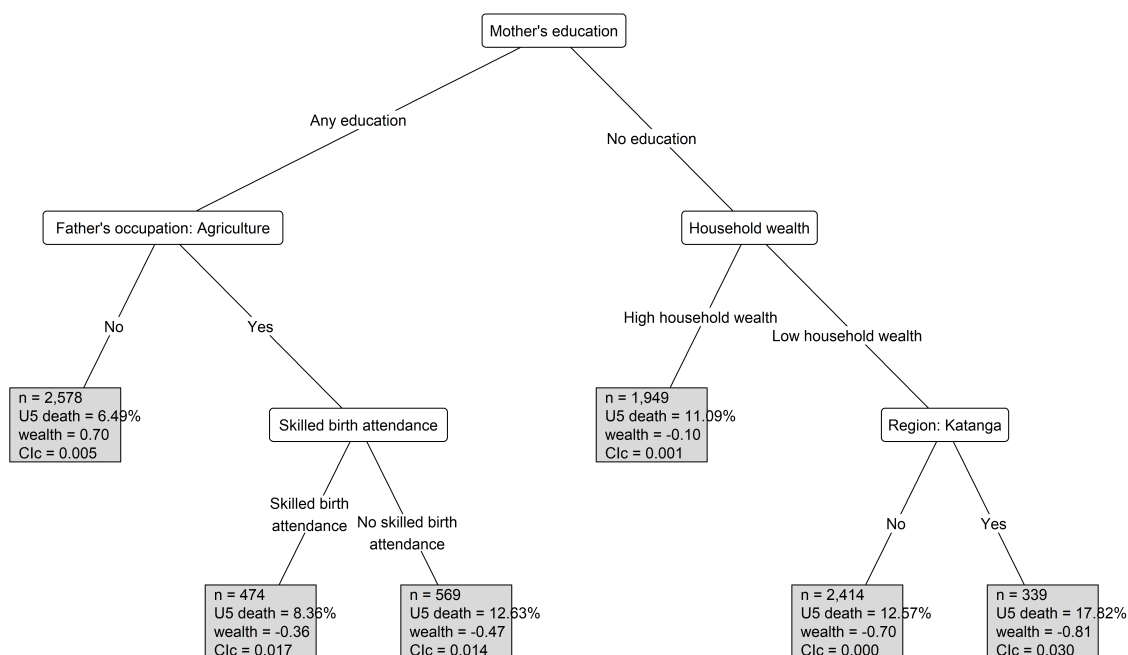
```



```

ci_report_tree_plot(
  fit = tree_models_by_type[["C1c"]],
  data = congo_model_dt,
  outcome_name = "deadu5_num",
  outcome_label = "U5 death",
  ci_type = "C1c",
  var_labels = congo_var_labels
)

```



6 Fit Lecturer Rpart Comparison Trees

The benchmark rpart trees are retained because they are easy to discuss beside the inequality-driven tree.

```

lecturer_rpart_models <- fit_ci_rpart_comparison(
  data = congo_model_dt,
  predictors = congo_predictors,
  outcome = "deadu5_num",
  rank = "wealth",
  weights = "sample_weight"
)
lecturer_rpart_summary <- summarise_ci_rpart_comparison(lecturer_rpart_models)

rpart_summary_out <- copy(lecturer_rpart_summary)[is_terminal == TRUE]

report_compact_table(
  rpart_summary_out,
  digits = 4,
  caption = "Terminal-node summaries for the lecturer rpart comparison trees"
)

```

Table 11: Terminal-node summaries for the lecturer rpart comparison trees

method	node	var	n	weight	deviance	mean_y	CI	CI_loss	is_terminal
simple_abs	16	<leaf>	907	926.0304	1.9375	0.0515	0.0021	0.0021	TRUE
simple_abs	17	<leaf>	1400	1571.0357	5.7339	0.0672	0.0036	0.0036	TRUE
simple_abs	18	<leaf>	43	18.3142	2.9358	0.1029	0.1603	0.1603	TRUE
simple_abs	38	<leaf>	14	5.4715	0.0000	0.0000	0.0000	0.0000	TRUE

simple_abs	78	<leaf>	7	2.8597	0.0000	0.0000	0.0000	0.0000	TRUE
simple_abs	79	<leaf>	120	54.3952	3.6348	0.2231	0.0668	0.0668	TRUE
simple_abs	10	<leaf>	68	86.5707	23.9450	0.1015	-0.2766	0.2766	TRUE
simple_abs	22	<leaf>	775	849.0024	5.0031	0.1014	-0.0059	0.0059	TRUE
simple_abs	23	<leaf>	77	108.0709	0.6296	0.1540	0.0058	0.0058	TRUE
simple_abs	6	<leaf>	2013	1948.8103	5.3891	0.1109	-0.0028	0.0028	TRUE
simple_abs	28	<leaf>	6	6.6534	0.0000	0.0000	0.0000	0.0000	TRUE
simple_abs	29	<leaf>	2512	2407.0755	0.3082	0.1260	-0.0001	0.0001	TRUE
simple_abs	30	<leaf>	162	145.1661	0.1906	0.1444	-0.0013	0.0013	TRUE
simple_abs	62	<leaf>	62	62.1253	0.4429	0.1359	0.0071	0.0071	TRUE
simple_abs	63	<leaf>	130	131.5099	8.6060	0.2354	-0.0654	0.0654	TRUE
robust_abs	8	<leaf>	1961	2025.0841	30.8769	0.0675	-0.0152	0.0152	TRUE
robust_abs	9	<leaf>	530	553.0227	8.0311	0.0552	-0.0145	0.0145	TRUE
robust_abs	10	<leaf>	374	474.3692	23.8352	0.0836	-0.0502	0.0502	TRUE
robust_abs	11	<leaf>	546	569.2748	15.4815	0.1263	-0.0272	0.0272	TRUE
robust_abs	6	<leaf>	2013	1948.8103	6.1032	0.1109	-0.0031	0.0031	TRUE
robust_abs	14	<leaf>	2518	2413.7289	1.4054	0.1257	0.0006	0.0006	TRUE
robust_abs	15	<leaf>	354	338.8012	14.2296	0.1782	-0.0420	0.0420	TRUE

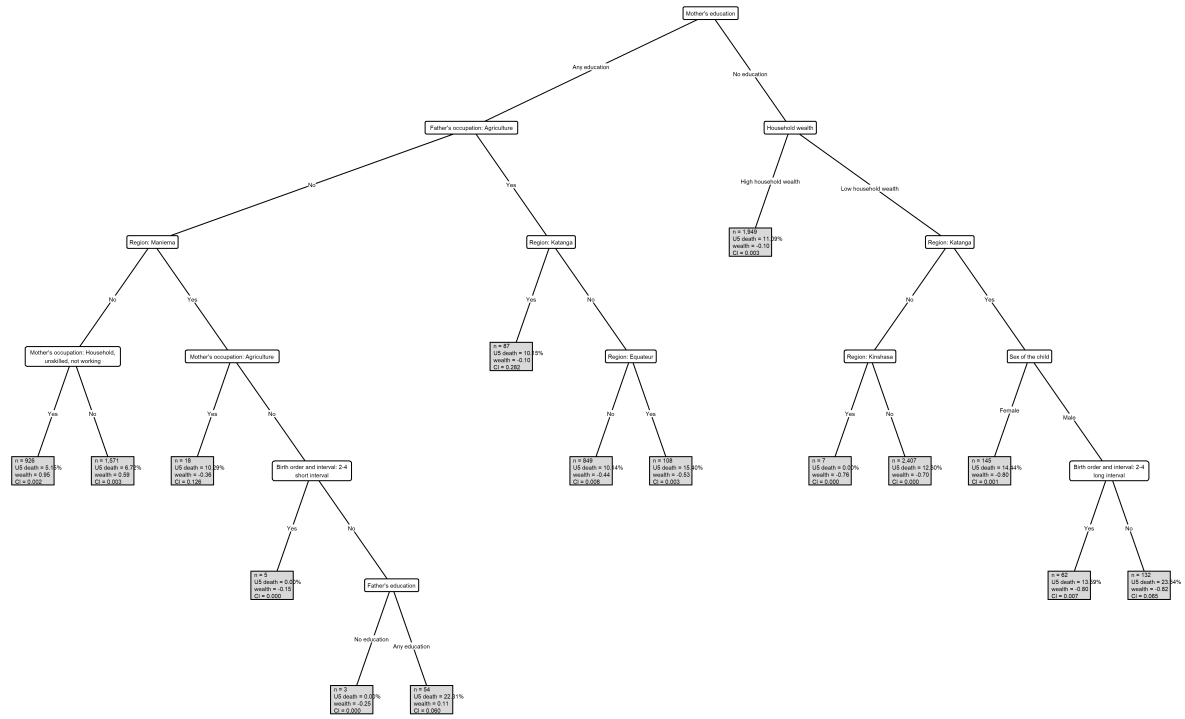
```

plot_labels <- c(
  simple_abs = "Lecturer rpart method 1: absolute CI impurity",
  robust_abs = "Lecturer rpart method 2: robust absolute CI impurity"
)
plot_fontsize <- c(simple_abs = 4.2, robust_abs = 4.6)

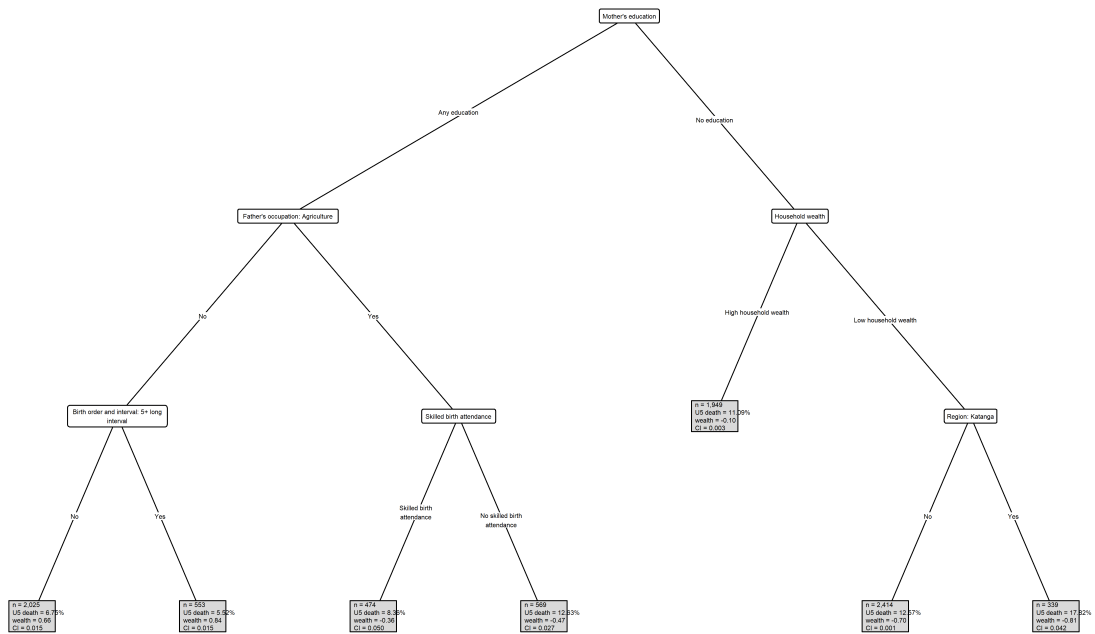
for (method_name in names(lecturer_rpart_models)) {
  cat("\n\n### ", plot_labels[[method_name]] %||% method_name, "\n\n", sep = "")
  ci_report_tree_plot(
    fit = lecturer_rpart_models[[method_name]],
    data = congo_model_dt,
    outcome_name = "deadu5_num",
    outcome_label = "U5 death",
    ci_type = "CI",
    var_labels = congo_var_labels,
    fontsize = plot_fontsize[[method_name]]
  )
}

```

6.0.1 Lecturer rpart method 1: absolute CI impurity



6.0.2 Lecturer rpart method 2: robust absolute CI impurity



7 Forest and SHAP Objects

These objects are the most expensive part of the generator, so their runtime is kept understandable by using the same row count and fold count set above. `rineq` does not expose a SHAP decomposition API, so the SHAP section uses a cross-validated `ranger` model and `shap_conc_decomp` for the four reported inequality criteria.

```
p <- length(congo_predictors)
forest_grid <- ci_tree_control_grid(
  minsplit = 500L,
  minbucket = 100L,
  minprob = 0.01,
  maxdepth = 15L,
  min_gain = 0.00001,
  min_relative_gain = c( 0.25, 0.30, 0.35, 0.40),
  mtry = c(10),
  ntree = 50L
)
```

```
forest_tuning <- run_forest_tuning_batches(
  formula = congo_ci_formula,
  data = congo_model_dt,
  forest_grid = forest_grid,
  criterion_types = criterion_types,
  tuning_metrics = ci_tuning_metrics,
  tuning_selection_metric = "relative_validation_gain",
  rank_name = "wealth",
  outcome_name = "deadu5_num",
  weights = congo_model_dt$sample_weight,
  folds = runtime_params$tuning_folds,
  workers = 4L,
  progress_steps = min(20L, nrow(forest_grid)),
  log_file = here("logs", "forest_tuning.log"),
  seed = 20260508,
  perturb = list(replace = FALSE, fraction = 0.7)
)
```

```
forest_best_by_type <- ci_select_best(
  forest_tuning,
  metric = "relative_validation_gain"
)
```

```
set.seed(20260509)
forest_models_by_type <- setNames(
  lapply(seq_len(nrow(forest_best_by_type)), function(i) {
    setting <- forest_best_by_type[i]
    ci_forest(
      formula = congo_ci_formula,
      data = congo_model_dt,
      rank_name = "wealth",
      outcome_name = "deadu5_num",
      weights = congo_model_dt$sample_weight,
      type = setting$type[1L],
      control = ci_control_from_row(setting, include_mtry = FALSE),
      ntree = as.integer(setting$ntree[1L]),
      mtry = as.integer(setting$mtry[1L]),
      perturb = list(replace = FALSE, fraction = 0.632)
    )
  })
)
```



```

    )
  }),
  forest_best_by_type$type
)
selected_forest_fit <- forest_models_by_type[[selected_tree_type]]
forest_variable_importance <- collect_variable_importance(forest_models_by_type)

```

```

forest_complexity_metrics <- collect_complexity_metrics(forest_tuning)
forest_min_relative_gain_path <- min_relative_gain_path(
  forest_complexity_metrics,
  selected = forest_best_by_type
)

```

```

forest_selection_table <- ci_fit_summary_table(
  forest_tuning,
  selected = forest_best_by_type,
  metrics = unique(forest_tuning$summary$metric),
  include_percent = FALSE
)
forest_selection_table <- merge(
  forest_selection_table,
  forest_best_by_type[, .(type, ntree, mtry)],
  by = "type",
  all.x = TRUE,
  sort = FALSE
)
forest_selection_table[
  forest_best_by_type,
  `:=`(
    mean_terminal_nodes = i.mean_terminal_nodes,
    mean_max_depth = i.maxdepth
  ),
  on = "type"
]
forest_selection_table[, selection_basis := paste(
  runtime_params$tuning_folds,
  "fold cross-validation"
)]

forest_fit_wide <- copy(forest_selection_table)
forest_fit_table <- ci_collect_metrics(
  forest_tuning,
  selected = forest_best_by_type,
  metric = unique(forest_tuning$summary$metric),
  format = "tidy",
  include_train = TRUE
)
forest_best_controls <- forest_best_by_type[
  ,
  .(
    type, ntree, mtry, minsplitted, minbucket,
    minprob, maxdepth,
    min_gain, min_relative_gain
  )
]

```

```

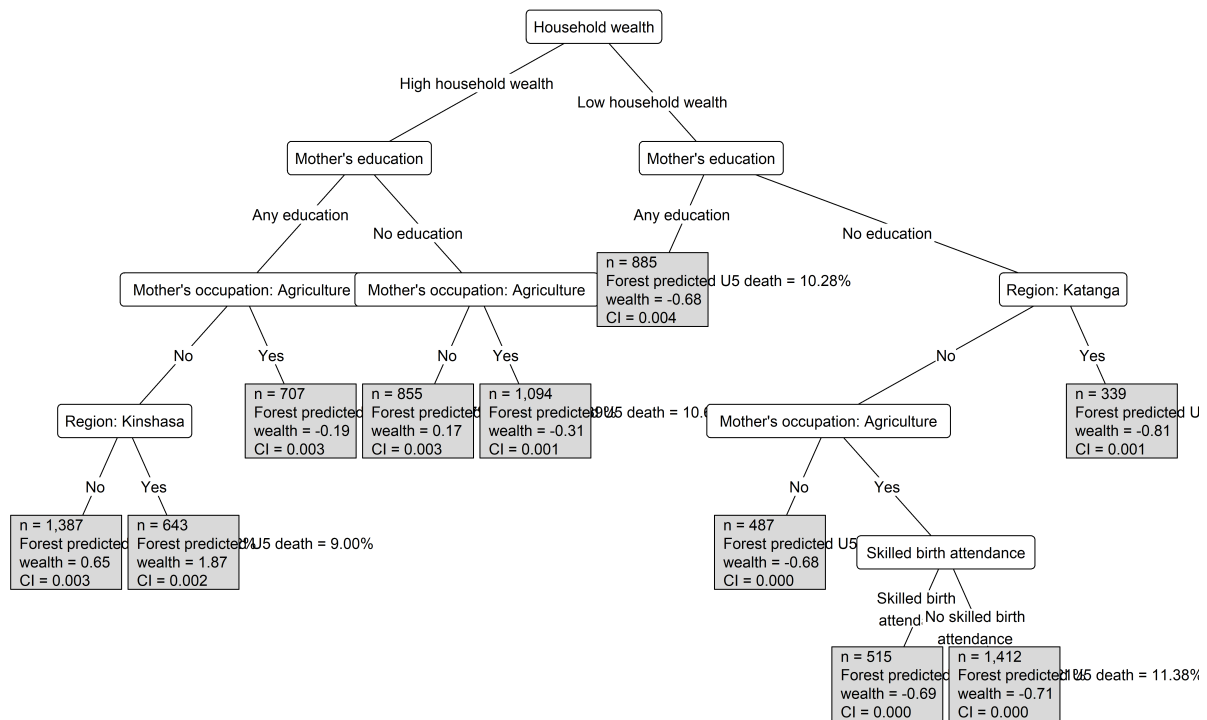
forest_surrogates_by_type <- setNames(
  lapply(seq_len(nrow(forest_best_by_type)), function(i) {
    setting <- copy(forest_best_by_type[i])
    setting$min_relative_gain <- 0.30
    setting$minsplit <- 1500L
    setting$minbucket <- 100L
    setting$maxdepth <- 5L
    ci_forest_surrogate(
      forest_fit = forest_models_by_type[[setting$type[1L]]],
      data = congo_model_dt,
      formula = congo_ci_formula,
      rank_name = "wealth",
      weights = congo_model_dt$sample_weight,
      type = setting$type[1L],
      control = ci_control_from_row(setting, include_mtry = FALSE),
      prediction_name = "forest_risk"
    )
  }),
  forest_best_by_type$type
)
selected_forest_surrogate <- forest_surrogates_by_type[[selected_tree_type]]
selected_forest_surrogate_summary <- as.data.table(
  ci_tree_terminal_summary(selected_forest_surrogate$fit)
)

```

```

ci_report_tree_plot(
  fit = forest_surrogates_by_type[["CI"]]$fit,
  data = forest_surrogates_by_type[["CI"]]$data,
  outcome_name = forest_surrogates_by_type[["CI"]]$prediction_name,
  outcome_label = "Forest predicted U5 death",
  ci_type = "CI",
  var_labels = congo_var_labels
)

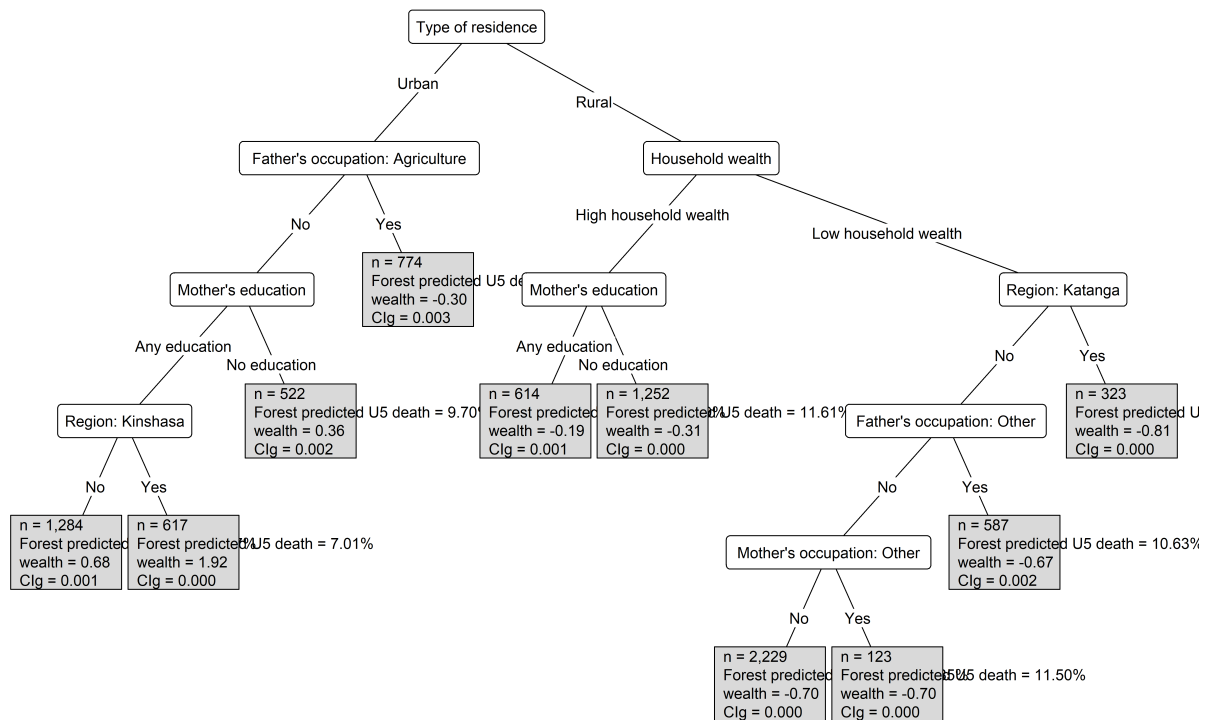
```



```

ci_report_tree_plot(
  fit = forest_surrogates_by_type[["CIg"]] $\$$ fit,
  data = forest_surrogates_by_type[["CIg"]] $\$$ data,
  outcome_name = forest_surrogates_by_type[["CIg"]] $\$$ prediction_name,
  outcome_label = "Forest predicted U5 death",
  ci_type = "CIg",
  var_labels = congo_var_labels
)

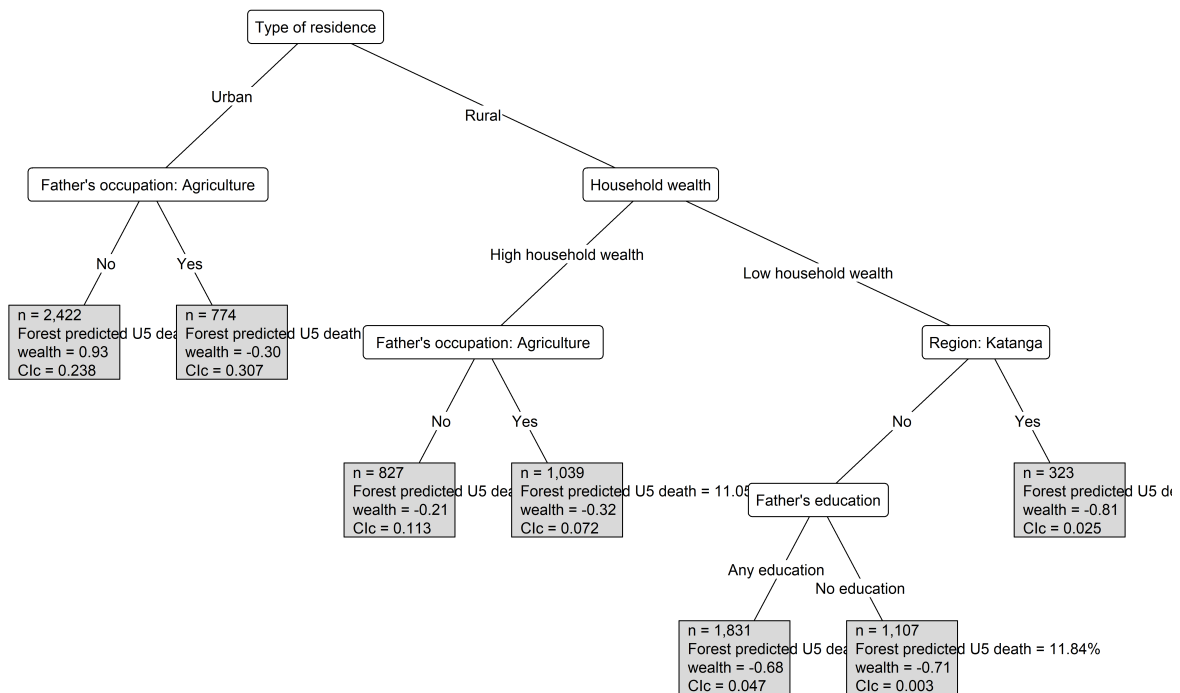
```



```

ci_report_tree_plot(
  fit = forest_surrogates_by_type[["CIG"]]$fit,
  data = forest_surrogates_by_type[["CIG"]]$data,
  outcome_name = forest_surrogates_by_type[["CIG"]]$prediction_name,
  outcome_label = "Forest predicted U5 death",
  ci_type = "CIG",
  var_labels = congo_var_labels
)

```



```
library(tidymodels)
ranger_analysis <- build_ranger_analysis_data(
  data = congo_model_dt,
  predictor_terms = congo_predictors,
  undersample = runtime_params$undersample,
  outcome_labels = outcome_labels
)
list2env(ranger_analysis, env = environment())
```

```
<environment: R_GlobalEnv>
```

```
ranger_recipe <- recipes::recipe(deadu5_factor ~ ., data = ranger_fit_df)
```

```
ranger_model_spec <- parsnip::rand_forest(
  mtry = tune::tune(),
  min_n = tune::tune(),
  trees = 500L
) |>
  parsnip::set_mode("classification") |>
  parsnip::set_engine("ranger", probability = TRUE, importance = "impurity")
```

```
ranger_resamples <- rsample::vfold_cv(
  ranger_fit_df,
  v = runtime_params$tuning_folds,
  strata = deadu5_factor
)
```

```
ranger_workflow <- workflows::workflow() |>
  workflows::add_model(ranger_model_spec) |>
  workflows::add_recipe(ranger_recipe) |>
  workflows::add_case_weights(sample_weight_case)
```

```
ranger_parameter_set <- hardhat::extract_parameter_set_dials(ranger_workflow) |>
  update(
    mtry = dials::mtry(range = c(10L, length(ranger_predictor_terms))),
    min_n = dials::min_n(range = c(200L, 2000L))
  )
ranger_grid <- dials::grid_regular(
  ranger_parameter_set,
  levels = c(mtry = 4, min_n = 3)
)
```

```
ranger_metric_set <- yardstick::metric_set(
  yardstick::roc_auc,
  yardstick::accuracy,
  yardstick::sens
)
ranger_tune_results <- tune::tune_grid(
  ranger_workflow,
  resamples = ranger_resamples,
  grid = ranger_grid,
  metrics = ranger_metric_set,
  control = tune::control_grid(save_pred = TRUE)
)
ranger_tuning <- as.data.table(tune::collect_metrics(ranger_tune_results))
```

```
report_compact_table(
  ranger_tuning,
  digits = 4,
  caption = "Predictive ranger cross-validation summary"
)
```

Table 12: Predictive ranger cross-validation summary

mtry	min_n	.metric	.estimator	mean	n	std_err	.config
10	200	accuracy	binary	0.6108	10	0.0090	pre0_mod01_post0
10	200	roc_auc	binary	0.6155	10	0.0111	pre0_mod01_post0
10	200	sens	binary	0.2774	10	0.0152	pre0_mod01_post0
10	1100	accuracy	binary	0.6072	10	0.0023	pre0_mod02_post0
10	1100	roc_auc	binary	0.6079	10	0.0108	pre0_mod02_post0
10	1100	sens	binary	0.0597	10	0.0089	pre0_mod02_post0
10	2000	accuracy	binary	0.5937	10	0.0003	pre0_mod03_post0
10	2000	roc_auc	binary	0.5000	10	0.0000	pre0_mod03_post0
10	2000	sens	binary	0.0000	10	0.0000	pre0_mod03_post0
16	200	accuracy	binary	0.6066	10	0.0084	pre0_mod04_post0
16	200	roc_auc	binary	0.6140	10	0.0104	pre0_mod04_post0
16	200	sens	binary	0.2900	10	0.0143	pre0_mod04_post0
16	1100	accuracy	binary	0.6100	10	0.0030	pre0_mod05_post0
16	1100	roc_auc	binary	0.6048	10	0.0110	pre0_mod05_post0
16	1100	sens	binary	0.0859	10	0.0120	pre0_mod05_post0
16	2000	accuracy	binary	0.5937	10	0.0003	pre0_mod06_post0
16	2000	roc_auc	binary	0.5000	10	0.0000	pre0_mod06_post0
16	2000	sens	binary	0.0000	10	0.0000	pre0_mod06_post0
22	200	accuracy	binary	0.6080	10	0.0098	pre0_mod07_post0
22	200	roc_auc	binary	0.6132	10	0.0105	pre0_mod07_post0

22	200	sens	binary	0.2923	10	0.0188	pre0_mod07_post0
22	1100	accuracy	binary	0.6132	10	0.0035	pre0_mod08_post0
22	1100	roc_auc	binary	0.6052	10	0.0108	pre0_mod08_post0
22	1100	sens	binary	0.1019	10	0.0138	pre0_mod08_post0
22	2000	accuracy	binary	0.5937	10	0.0003	pre0_mod09_post0
22	2000	roc_auc	binary	0.5000	10	0.0000	pre0_mod09_post0
22	2000	sens	binary	0.0000	10	0.0000	pre0_mod09_post0
29	200	accuracy	binary	0.6090	10	0.0106	pre0_mod10_post0
29	200	roc_auc	binary	0.6126	10	0.0103	pre0_mod10_post0
29	200	sens	binary	0.3072	10	0.0162	pre0_mod10_post0
29	1100	accuracy	binary	0.6123	10	0.0042	pre0_mod11_post0
29	1100	roc_auc	binary	0.6053	10	0.0108	pre0_mod11_post0
29	1100	sens	binary	0.1100	10	0.0105	pre0_mod11_post0
29	2000	accuracy	binary	0.5937	10	0.0003	pre0_mod12_post0
29	2000	roc_auc	binary	0.5000	10	0.0000	pre0_mod12_post0
29	2000	sens	binary	0.0000	10	0.0000	pre0_mod12_post0

```
selected_ranger_setting <- tune::select_best(ranger_tune_results, metric = "roc_auc")
```

```
report_compact_table(
  as.data.table(as.data.frame(selected_ranger_setting)),
  digits = 4,
  caption = "Selected predictive ranger setting"
)
```

Table 13: Selected predictive ranger setting

mtry	min_n	.config
10	200	pre0_mod01_post0

```
final_ranger_workflow <- tune::finalize_workflow(ranger_workflow, selected_ranger_setting)
predictive_forest_fit <- workflows::fit(final_ranger_workflow, data = ranger_fit_df)
```

```
predict_ranger_risk <- function(object, newdata) {
  predict(object, new_data = as.data.frame(newdata), type = "prob")$.pred_Died
}
```

```
shap_rows <- sort(sample.int(nrow(ranger_fit_df), max(shap_n, nrow(ranger_fit_df))))
shap_x_background <- ranger_fit_df[, ranger_predictor_terms, drop = FALSE]
shap_x_eval <- ranger_fit_df[shap_rows, ranger_predictor_terms, drop = FALSE]
shap_prediction <- as.numeric(predict_ranger_risk(predictive_forest_fit, shap_x_eval))
```

```
selected_forest_shap <- fastshap::explain(
  object = predictive_forest_fit,
  X = shap_x_background,
  pred_wrapper = predict_ranger_risk,
  newdata = shap_x_eval,
  nsim = shap_nsim,
  adjust = shap_nsim > 1L
)
colnames(selected_forest_shap) <- canonicalize_model_terms(
```

```

    colnames(selected_forest_shap),
    congo_predictors
)

shap_feature_values <- copy(as.data.table(shap_x_eval))
shap_feature_values[, observation_id := seq_len(.N)]
selected_forest_shap_dt <- as.data.table(selected_forest_shap)
selected_forest_shap_dt[, observation_id := seq_len(.N)]

shap_summary_long <- melt(
  selected_forest_shap_dt,
  id.vars = "observation_id",
  variable.name = "feature",
  value.name = "shap_value",
  variable.factor = FALSE
)
shap_value_long <- melt(
  shap_feature_values,
  id.vars = "observation_id",
  variable.name = "feature",
  value.name = "feature_value",
  variable.factor = FALSE
)
shap_summary_long <- merge(
  shap_summary_long,
  shap_value_long,
  by = c("observation_id", "feature"),
  all.x = TRUE,
  sort = FALSE
)
shap_summary_long[
  ,
  feature_value_numeric := suppressWarnings(as.numeric(as.character(feature_value)))
]
shap_summary_long[
  is.na(feature_value_numeric),
  feature_value_numeric := as.numeric(factor(feature_value)) - 1
]
shap_summary_long[
  ,
  feature_value_scaled := {
    range_value <- range(feature_value_numeric, na.rm = TRUE)
    fifelse(
      rep(all(is.finite(range_value)) && diff(range_value) > .Machine$double.eps, .N),
      (feature_value_numeric - range_value[1L]) / diff(range_value),
      0.5
    )
  },
  by = feature
]
shap_summary_long[, feature_label := format_variable_label(feature)]
shap_summary_long[
  ,
  mean_abs_shap := mean(abs(shap_value), na.rm = TRUE),
  by = feature
]
data.table::setcolorder(
  shap_summary_long,

```



```

c(
  "feature",
  "feature_label",
  "observation_id",
  "shap_value",
  "feature_value",
  "feature_value_numeric",
  "feature_value_scaled",
  "mean_abs_shap"
)
)

```

```

shap_ci_decomp <- shap_conc_decomp(
  shap = selected_forest_shap,
  rank = ranger_dt$wealth[shap_rows],
  type = "CI",
  prediction = shap_prediction,
  weights = ranger_dt$sample_weight[shap_rows]
)

```

```

shap_ci_diag <- as.data.table(shap_ci_decomp$diagnostics)
shap_ci_contrib <- as.data.table(shap_ci_decomp$contributions)[order(pct_contribution)]
report_compact_table(
  shap_ci_diag,
  digits = 4,
  caption = "SHAP decomposition diagnostics: CI"
)

```

Table 14: SHAP decomposition diagnostics

n	weight_sum	type	mean_prediction	concentration_index	signed_concentration_index
2146	2148.538	CI	0.3999	0.0765	-0.0765

```

report_compact_table(
  shap_ci_contrib,
  digits = 4,
  caption = "SHAP decomposition contributions: CI"
)

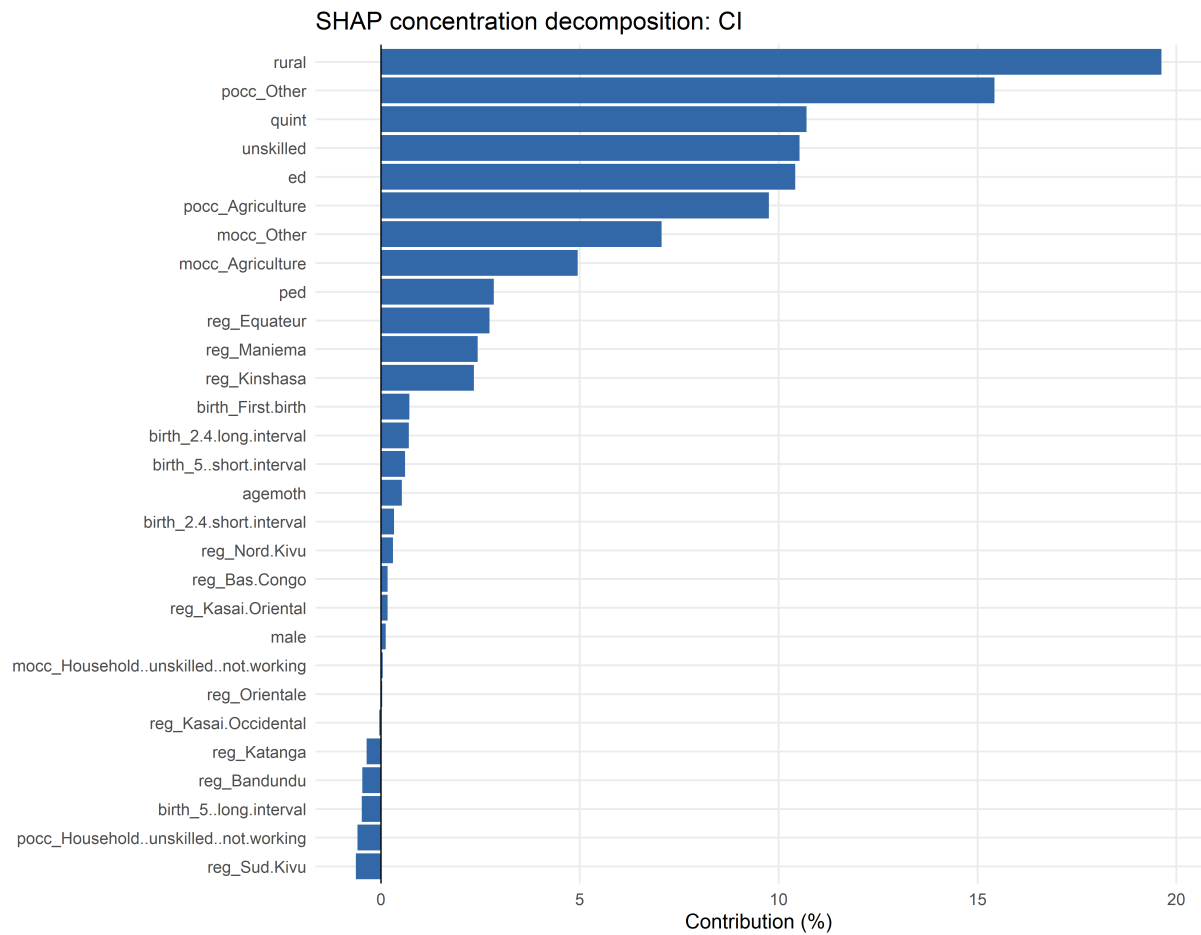
```

Table 15: SHAP decomposition contributions: CI

	feature	D_k_SHAP	pct_contribution	abs_contribution
	reg_Sud.Kivu	-0.0005	-0.6304	0.0005
pocc_Household..unskilled..not.working		-0.0004	-0.5874	0.0004
	birth_5..long.interval	-0.0004	-0.4772	0.0004
	reg_Bandundu	-0.0004	-0.4686	0.0004
	reg_Katanga	-0.0003	-0.3590	0.0003
	reg_Kasai.Occidental	0.0000	-0.0370	0.0000
	reg_Orientale	0.0000	0.0379	0.0000
mocc_Household..unskilled..not.working		0.0000	0.0498	0.0000
	male	0.0001	0.1257	0.0001
	reg_Kasai.Oriental	0.0001	0.1687	0.0001

reg_Bas.Congo	0.0001	0.1703	0.0001
reg_Nord.Kivu	0.0002	0.3066	0.0002
birth_2.4.short.interval	0.0003	0.3313	0.0003
agemoth	0.0004	0.5298	0.0004
birth_5..short.interval	0.0005	0.6096	0.0005
birth_2.4.long.interval	0.0005	0.7063	0.0005
birth_First.birth	0.0006	0.7204	0.0006
reg_Kinshasa	0.0018	2.3398	0.0018
reg_Maniema	0.0019	2.4334	0.0019
reg_Equateur	0.0021	2.7316	0.0021
ped	0.0022	2.8373	0.0022
mocc_Agriculture	0.0038	4.9467	0.0038
mocc_Other	0.0054	7.0601	0.0054
pocc_Agriculture	0.0075	9.7533	0.0075
ed	0.0080	10.4208	0.0080
unskilled	0.0081	10.5280	0.0081
quint	0.0082	10.7023	0.0082
pocc_Other	0.0118	15.4270	0.0118
rural	0.0150	19.6233	0.0150

```
ggplot(shap_ci_contrib, aes(x = pct_contribution, y = reorder(feature, pct_contribution))) +
  geom_col(fill = "#3268A8") +
  geom_vline(xintercept = 0, color = "black", linewidth = 0.4) +
  labs(x = "Contribution (%)", y = NULL, title = "SHAP concentration decomposition: CI") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank())
```



```
shap_cig_decomp <- shap_conc_decomp(
  shap = selected_forest_shap,
  rank = ranger_dt$wealth[shap_rows],
  type = "CIg",
  prediction = shap_prediction,
  weights = ranger_dt$sample_weight[shap_rows]
)
```

```
shap_cig_diag <- as.data.table(shap_cig_decomp$diagnostics)
shap_cig_contrib <- as.data.table(shap_cig_decomp$contributions)[order(pct_contribution)]
report_compact_table(
  shap_cig_diag,
  digits = 4,
  caption = "SHAP decomposition diagnostics: CIg"
)
```

Table 16: SHAP decomposition

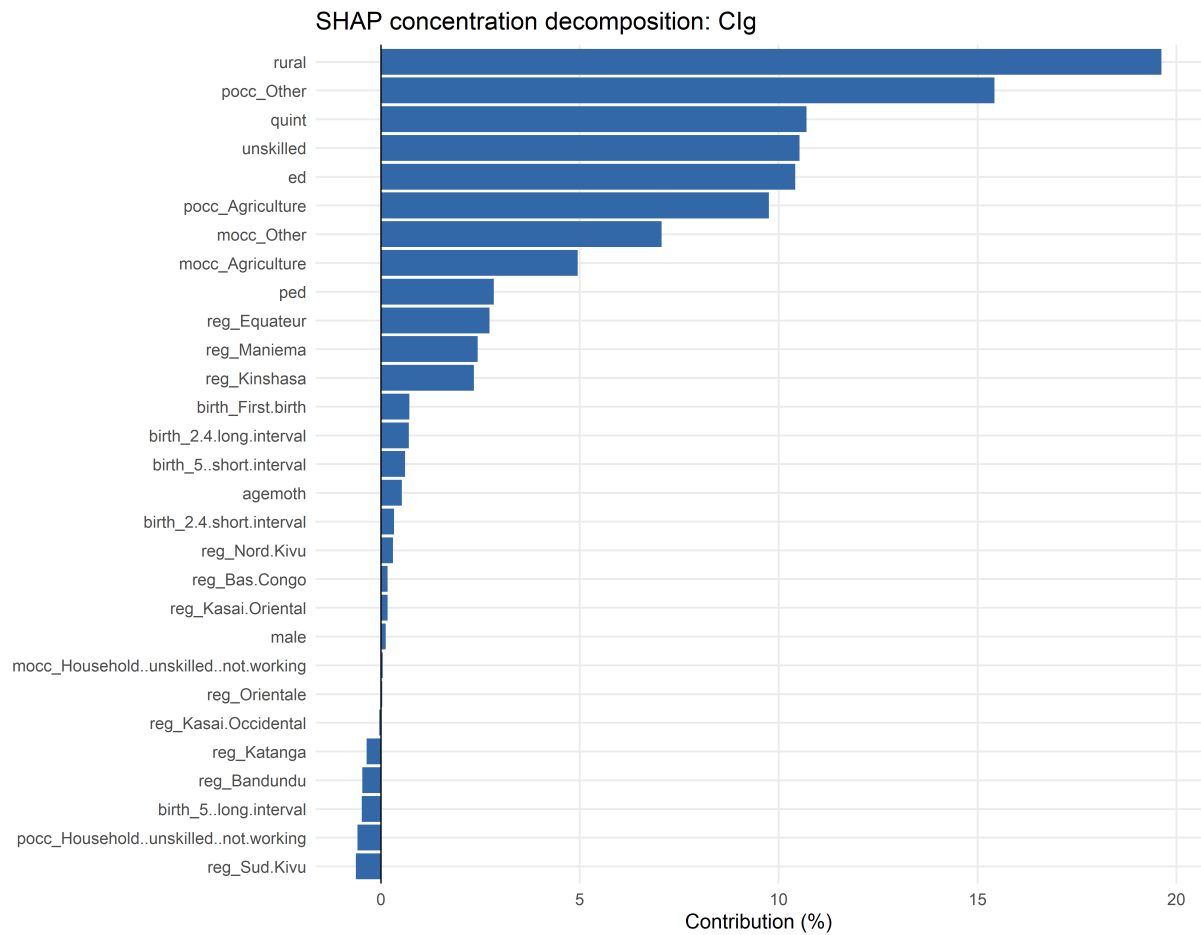
n	weight_sum	type	mean_prediction	concentration_index	signed_concentration_index
2146	2148.538	CIg	0.3999	0.0306	-0.0306

```
report_compact_table(
  shap_cig_contrib,
  digits = 4,
  caption = "SHAP decomposition contributions: CIg"
)
```

Table 17: SHAP decomposition contributions: CIg

	feature	D_k_SHAP	pct_contribution	abs_contribution
	reg_Sud.Kivu	-0.0002	-0.6304	0.0002
pocc_Household..unskilled..not.working		-0.0002	-0.5874	0.0002
	birth_5..long.interval	-0.0001	-0.4772	0.0001
	reg_Bandundu	-0.0001	-0.4686	0.0001
	reg_Katanga	-0.0001	-0.3590	0.0001
	reg_Kasai.Occidental	0.0000	-0.0370	0.0000
	reg_Orientale	0.0000	0.0379	0.0000
mocc_Household..unskilled..not.working		0.0000	0.0498	0.0000
	male	0.0000	0.1257	0.0000
	reg_Kasai.Oriental	0.0001	0.1687	0.0001
	reg_Bas.Congo	0.0001	0.1703	0.0001
	reg_Nord.Kivu	0.0001	0.3066	0.0001
birth_2.4.short.interval		0.0001	0.3313	0.0001
	agemoth	0.0002	0.5298	0.0002
birth_5..short.interval		0.0002	0.6096	0.0002
birth_2.4.long.interval		0.0002	0.7063	0.0002
	birth_First.birth	0.0002	0.7204	0.0002
	reg_Kinshasa	0.0007	2.3398	0.0007
	reg_Maniema	0.0007	2.4334	0.0007
	reg_Equateur	0.0008	2.7316	0.0008
	ped	0.0009	2.8373	0.0009
mocc_Agriculture		0.0015	4.9467	0.0015
	mocc_Other	0.0022	7.0601	0.0022
pocc_Agriculture		0.0030	9.7533	0.0030
	ed	0.0032	10.4208	0.0032
	unskilled	0.0032	10.5280	0.0032
	quint	0.0033	10.7023	0.0033
pocc_Other		0.0047	15.4270	0.0047
	rural	0.0060	19.6233	0.0060

```
ggplot(shap_cig_contrib, aes(x = pct_contribution, y = reorder(feature, pct_contribution))) +
  geom_col(fill = "#3268A8") +
  geom_vline(xintercept = 0, color = "black", linewidth = 0.4) +
  labs(x = "Contribution (%)", y = NULL, title = "SHAP concentration decomposition: CIg") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank())
```



```
shap_cic_decomp <- shap_conc_decomp(
  shap = selected_forest_shap,
  rank = ranger_dt$wealth[shap_rows],
  type = "CIc",
  prediction = shap_prediction,
  weights = ranger_dt$sample_weight[shap_rows]
)
```

```
shap_cic_diag <- as.data.table(shap_cic_decomp$diagnostics)
shap_cic_contrib <- as.data.table(shap_cic_decomp$contributions)[order(pct_contribution)]
report_compact_table(
  shap_cic_diag,
  digits = 4,
  caption = "SHAP decomposition diagnostics: CIc"
)
```

Table 18: SHAP decomposition

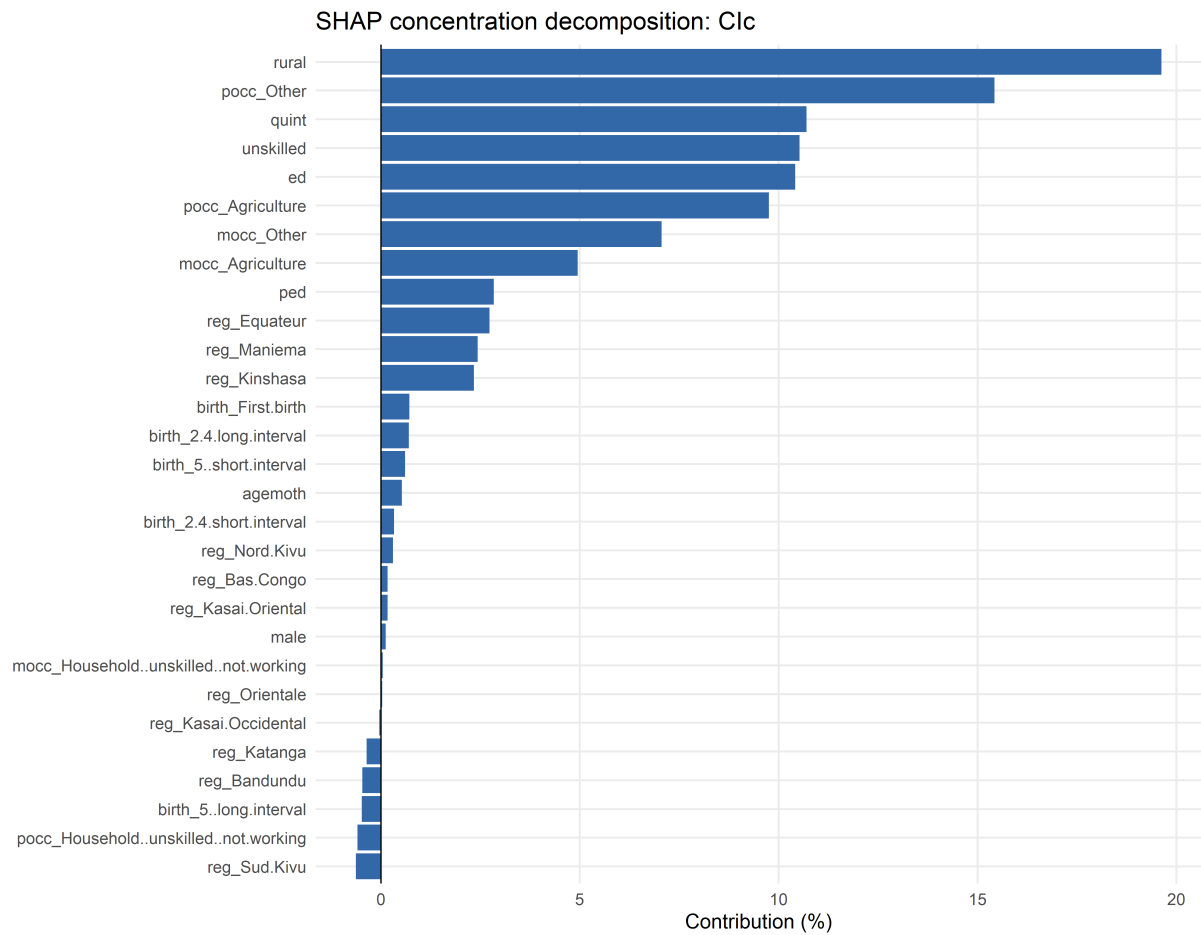
n	weight_sum	type	mean_prediction	concentration_index	signed_concentration_index
2146	2148.538	CIc	0.3999	0.2268	-0.2268

```
report_compact_table(
  shap_cic_contrib,
  digits = 4,
  caption = "SHAP decomposition contributions: CIc"
)
```

Table 19: SHAP decomposition contributions: C1c

feature	D_k_SHAP	pct_contribution	abs_contribution
reg_Sud.Kivu	-0.0014	-0.6304	0.0014
pocc_Household..unskilled..not.working	-0.0013	-0.5874	0.0013
birth_5..long.interval	-0.0011	-0.4772	0.0011
reg_Bandundu	-0.0011	-0.4686	0.0011
reg_Katanga	-0.0008	-0.3590	0.0008
reg_Kasai.Occidental	-0.0001	-0.0370	0.0001
reg_Orientale	0.0001	0.0379	0.0001
mocc_Household..unskilled..not.working	0.0001	0.0498	0.0001
male	0.0003	0.1257	0.0003
reg_Kasai.Oriental	0.0004	0.1687	0.0004
reg_Bas.Congo	0.0004	0.1703	0.0004
reg_Nord.Kivu	0.0007	0.3066	0.0007
birth_2.4.short.interval	0.0008	0.3313	0.0008
agemoth	0.0012	0.5298	0.0012
birth_5..short.interval	0.0014	0.6096	0.0014
birth_2.4.long.interval	0.0016	0.7063	0.0016
birth_First.birth	0.0016	0.7204	0.0016
reg_Kinshasa	0.0053	2.3398	0.0053
reg_Maniema	0.0055	2.4334	0.0055
reg_Equateur	0.0062	2.7316	0.0062
ped	0.0064	2.8373	0.0064
mocc_Agriculture	0.0112	4.9467	0.0112
mocc_Other	0.0160	7.0601	0.0160
pocc_Agriculture	0.0221	9.7533	0.0221
ed	0.0236	10.4208	0.0236
unskilled	0.0239	10.5280	0.0239
quint	0.0243	10.7023	0.0243
pocc_Other	0.0350	15.4270	0.0350
rural	0.0445	19.6233	0.0445

```
ggplot(shap_cic_contrib, aes(x = pct_contribution, y = reorder(feature, pct_contribution))) +
  geom_col(fill = "#3268A8") +
  geom_vline(xintercept = 0, color = "black", linewidth = 0.4) +
  labs(x = "Contribution (%)", y = NULL, title = "SHAP concentration decomposition: C1c") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank())
```



```
shap_l_decomp <- shap_conc_decomp(
  shap = selected_forest_shap,
  rank = ranger_dt$wealth[shap_rows],
  type = "L",
  prediction = shap_prediction,
  weights = ranger_dt$sample_weight[shap_rows]
)
```

```
shap_l_diag <- as.data.table(shap_l_decomp$diagnostics)
shap_l_contrib <- as.data.table(shap_l_decomp$contributions)[order(pct_contribution)]
report_compact_table(
  shap_l_diag,
  digits = 4,
  caption = "SHAP decomposition diagnostics: L"
)
```

Table 20: SHAP decomposition

n	weight_sum	type	mean_prediction	concentration_index	signed_concentration_index
2146	2148.538	L	0.3999	0.309	0.309

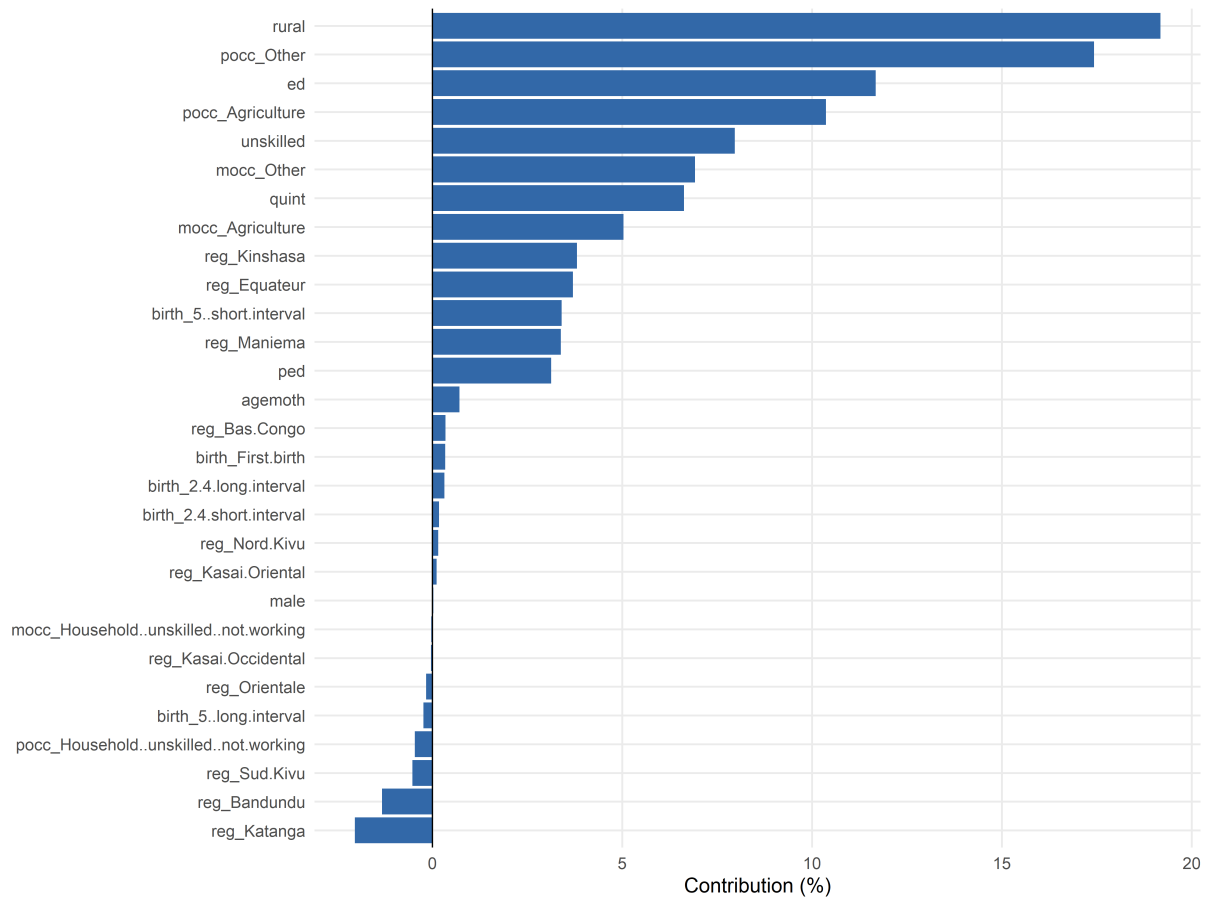
```
report_compact_table(
  shap_l_contrib,
  digits = 4,
  caption = "SHAP decomposition contributions: L"
)
```

Table 21: SHAP decomposition contributions: L

feature	D_k_SHAP	pct_contribution	abs_contribution
reg_Katanga	-0.0063	-2.0434	0.0063
reg_Bandundu	-0.0041	-1.3275	0.0041
reg_Sud.Kivu	-0.0016	-0.5216	0.0016
pocc_Household..unskilled..not.working	-0.0014	-0.4646	0.0014
birth_5..long.interval	-0.0007	-0.2325	0.0007
reg_Orientale	-0.0005	-0.1657	0.0005
reg_Kasai.Occidental	-0.0001	-0.0335	0.0001
mocc_Household..unskilled..not.working	-0.0001	-0.0305	0.0001
male	0.0001	0.0252	0.0001
reg_Kasai.Oriental	0.0003	0.1098	0.0003
reg_Nord.Kivu	0.0005	0.1571	0.0005
birth_2.4.short.interval	0.0005	0.1770	0.0005
birth_2.4.long.interval	0.0010	0.3210	0.0010
birth_First.birth	0.0011	0.3407	0.0011
reg_Bas.Congo	0.0011	0.3463	0.0011
agemoth	0.0022	0.7169	0.0022
ped	0.0097	3.1286	0.0097
reg_Maniema	0.0105	3.3868	0.0105
birth_5..short.interval	0.0105	3.4026	0.0105
reg_Equateur	0.0114	3.7042	0.0114
reg_Kinshasa	0.0118	3.8114	0.0118
mocc_Agriculture	0.0155	5.0305	0.0155
quint	0.0205	6.6301	0.0205
mocc_Other	0.0214	6.9170	0.0214
unskilled	0.0246	7.9684	0.0246
pocc_Agriculture	0.0320	10.3641	0.0320
ed	0.0361	11.6782	0.0361
pocc_Other	0.0539	17.4301	0.0539
rural	0.0593	19.1734	0.0593

```
ggplot(shap_l_contrib, aes(x = pct_contribution, y = reorder(feature, pct_contribution))) +
  geom_col(fill = "#3268A8") +
  geom_vline(xintercept = 0, color = "black", linewidth = 0.4) +
  labs(x = "Contribution (%)", y = NULL, title = "SHAP concentration decomposition: L") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank())
```


SHAP concentration decomposition: L



```
selected_shap_contributions <- rbindlist(list(
  as.data.table(shap_ci_decomp$contributions)[, criterion := "CI"],
  as.data.table(shap_cig_decomp$contributions)[, criterion := "CIg"],
  as.data.table(shap_cic_decomp$contributions)[, criterion := "CIc"],
  as.data.table(shap_l_decomp$contributions)[, criterion := "L"]
), fill = TRUE)
selected_shap_contributions[, feature := canonicalize_model_terms(feature, congo_predictors)]
data.table::setcolororder(selected_shap_contributions, c("criterion", setdiff(names(selected_shap_contributions), "criterion")))

selected_shap_diagnostics <- rbindlist(list(
  as.data.table(shap_ci_decomp$diagnostics)[, criterion := "CI"],
  as.data.table(shap_cig_decomp$diagnostics)[, criterion := "CIg"],
  as.data.table(shap_cic_decomp$diagnostics)[, criterion := "CIc"],
  as.data.table(shap_l_decomp$diagnostics)[, criterion := "L"]
), fill = TRUE)
data.table::setcolororder(selected_shap_diagnostics, c("criterion", setdiff(names(selected_shap_diagnostics), "criterion")))

forest_importance_out <- copy(forest_variable_importance)
forest_importance_out[, variable_label := fifelse(
  variable %in% names(congo_var_labels),
  congo_var_labels[variable],
  variable
)]

report_compact_table(
  forest_importance_out,
  digits = 4,
```

```
caption = "Forest variable importance preview"
)
```

Table 22: Forest variable importance preview

crit	variable	importance	rank	
CI	ed	0.0114	1	
CI	quint	0.0103	2	
CI	mocc_Agriculture	0.0052	3	Mother's
CI	pocc_Other	0.0029	4	Fat
CI	pocc_Agriculture	0.0025	5	Father's
CI	reg_Kinshasa	0.0018	6	
CI	unskilled	0.0009	7	
CI	rural	0.0008	8	
CI	birth_2.4.short.interval	0.0005	9	Birth order and int
CI	reg_Bandundu	0.0003	10	
CI	reg_Katanga	0.0003	11	
CI	mocc_Household..unskilled..not.working	0.0003	12	Mother's occupation: Household
CIc	ed	0.0078	1	
CIc	quint	0.0060	2	
CIc	rural	0.0044	3	
CIc	pocc_Agriculture	0.0039	4	Father's
CIc	mocc_Agriculture	0.0028	5	Mother's
CIc	reg_Katanga	0.0012	6	
CIc	pocc_Other	0.0010	7	Fat
CIc	mocc_Other	0.0002	8	Mot
CIc	reg_Kinshasa	0.0002	9	
CIc	reg_Bandundu	0.0002	10	
CIc	mocc_Household..unskilled..not.working	0.0001	11	Mother's occupation: Household
CIg	ed	0.0025	1	
CIg	rural	0.0018	2	
CIg	pocc_Agriculture	0.0012	3	Father's
CIg	quint	0.0011	4	
CIg	mocc_Agriculture	0.0009	5	Mother's
CIg	pocc_Other	0.0007	6	Fat
CIg	reg_Kinshasa	0.0005	7	
CIg	reg_Katanga	0.0003	8	
CIg	unskilled	0.0001	9	
CIg	mocc_Household..unskilled..not.working	0.0001	10	Mother's occupation: Household
CIg	pocc_Household..unskilled..not.working	0.0001	11	Father's occupation: Household
CIg	mocc_Other	0.0001	12	Mot
L	rural	0.0664	1	
L	ed	0.0393	2	
L	mocc_Agriculture	0.0292	3	Mother's
L	pocc_Agriculture	0.0201	4	Father's
L	quint	0.0094	5	
L	pocc_Other	0.0069	6	Fat
L	reg_Kinshasa	0.0064	7	
L	mocc_Household..unskilled..not.working	0.0003	8	Mother's occupation: Household
L	reg_Katanga	0.0001	9	

8 Compare Methods

Once the regression, tree, forest, and SHAP objects exist, the next step is to rank variables and assemble the comparison tables used in the main report.

```
linear_rank <- classical_decomposition_table[
  criterion == "CI"
][
  order(-abs(regression_pct_contribution)),
  .(variable, method = "Linear decomposition", rank = seq_len(.N))
]
tree_rank <- rank_from_named_vector(selected_tree_fit$variable.importance, "Selected CI-tree")
forest_rank <- rank_from_named_vector(selected_forest_fit$variable.importance, "Selected CI-forest")
shap_rank <- selected_shap_contributions[
  criterion == "CI"
][
  order(-abs_contribution),
  .(variable = feature, method = "Selected forest SHAP", rank = seq_len(.N))
]
importance_long <- rbindlist(list(linear_rank, tree_rank, forest_rank, shap_rank), fill = TRUE)

selected_variables <- classical_decomposition_table[
  criterion == "CI"
][
  order(-abs(regression_pct_contribution))
]$variable[seq_len(min(15L, classical_decomposition_table[criterion == "CI", .N]))]
selected_variables <- selected_variables[!is.na(selected_variables) & nzchar(selected_variables)]

method_comparison_table <- data.table(variable = selected_variables)
method_comparison_table[, variable_label := format_variable_label(variable)]
regression_wide <- dcast(
  classical_decomposition_table[
    criterion %in% c("CI", "CIg", "CIc"),
    .(variable, criterion, regression_pct_contribution)
  ],
  variable ~ criterion,
  value.var = "regression_pct_contribution"
)
method_comparison_table <- merge(
  method_comparison_table,
  regression_wide[
    ,
    .(
      variable,
      regression_ci_pct_contribution = CI,
      regression_cig_pct_contribution = CIg,
      regression_cic_pct_contribution = CIc
    )
  ],
  by = "variable",
  all.x = TRUE
)
shap_wide <- dcast(
  selected_shap_contributions[
```

```

    criterion %in% criterion_types,
    .(variable = feature, criterion, pct_contribution)
  ],
  variable ~ criterion,
  value.var = "pct_contribution"
)
shap_wide <- shap_wide[
  ,
  .(
    variable,
    forest_shap_ci_pct_contribution = CI,
    forest_shap_cig_pct_contribution = CIg,
    forest_shap_cic_pct_contribution = CIC,
    forest_shap_l_pct_contribution = L
  )
]
method_comparison_table <- merge(
  method_comparison_table,
  shap_wide,
  by = "variable",
  all.x = TRUE
)
method_comparison_table <- method_comparison_table[
  order(-abs(regression_ci_pct_contribution), -abs(forest_shap_ci_pct_contribution))
]

importance_wide <- dcast(
  importance_long,
  rank ~ method,
  value.var = "variable",
  fun.aggregate = function(x) paste(unique(x), collapse = ", ")
)
for (col in setdiff(names(importance_wide), "rank")) {
  importance_wide[, (col) := format_variable_label(get(col))]
}
importance_wide <- importance_wide[rank <= 10L]

report_compact_table(
  method_comparison_table,
  digits = 3,
  caption = "Selected percentage-contribution comparison table"
)

```

variable	variable_label	regression
ed	Mother's education	
pocc_Agriculture	Father's occupation: Agriculture	
mocc_Agriculture	Mother's occupation: Agriculture	
quint	Household wealth	
rural	Type of residence	
unskilled	Skilled birth attendance	
reg_Equateur	Region: Equateur	
pocc_Household..unskilled..not.working	Father's occupation: Household, unskilled, not working	
reg_Kinshasa	Region: Kinshasa	

reg_Sud.Kivu	Region: Sud-Kivu
reg_Kasai.Oriental	Region: Kasai Oriental
ped	Father's education
reg_Maniema	Region: Maniema
reg_Orientale	Region: Orientale
reg_Bas.Congo	Region: Bas-Congo

```
report_compact_table(
  importance_wide,
  caption = "Ranked variables across decomposition and tree-based methods"
)
```

Table 24: Ranked variables across decomposition and

rank	Linear decomposition	Selected
1	Mother's education	Type of residence
2	Father's occupation: Agriculture	Mother's occupation: Agriculture
3	Mother's occupation: Agriculture	Mother's occupation: Agriculture
4	Household wealth	Father's occupation: Agriculture
5	Type of residence	Household wealth
6	Skilled birth attendance	Father's occupation: Agriculture
7	Region: Equateur	Region: Equateur
8	Father's occupation: Household, unskilled, not working	Mother's occupation: Household, unskilled, not working
9	Region: Kinshasa	Region: Kinshasa
10	Region: Sud-Kivu	Father's occupation: Household, unskilled, not working

9 Save Results Objects

The last chunk assembles the full object list and writes it to disk for the main report to consume.

```
drc_results_objects <- list(
  generation_params = runtime_params,
  source_data_file = source_data_file,
  paper_reference_file = paper_reference_file,
  outcome_labels = outcome_labels,
  congo_model_dt = congo_model_dt,
  raw_congo_model_dt = raw_congo_model_dt,
  congo_var_labels = congo_var_labels,
  raw_congo_var_labels = raw_congo_var_labels,
  congo_predictors = congo_predictors,
  raw_congo_predictors = raw_congo_predictors,
  indicator_level_map = indicator_level_map,
  congo_ci_formula = congo_ci_formula,
  data_summary = drc_data_summary_table,
  outcome_distribution = outcome_plot_dt,
  candidate_predictors = candidate_predictor_table,
  root_ci = drc_root_ci_table,
  regression_model_summary = regression_model_summary,
  classical_decomposition = classical_decomposition_table,
  classical_decomposition_grouped = classical_decomposition_grouped,
  published_drc_table3_reference = published_drc_table3_reference,
  published_drc_table3_comparison = published_drc_table3_comparison,
```

```

rinq_summary = rinq_summary,
tree_tuning = tree_tuning,
tree_best_by_type = tree_best_by_type,
tree_best_controls = tree_best_controls,
tree_selection = tree_selection_table,
tree_fit_table = tree_fit_table,
tree_complexity_metrics = tree_complexity_metrics,
tree_min_relative_gain_path = tree_min_relative_gain_path,
tree_models_by_type = tree_models_by_type,
selected_tree_type = selected_tree_type,
selected_tree_fit = selected_tree_fit,
selected_tree_summary = selected_tree_summary,
all_tree_terminal_summaries = all_tree_terminal_summaries,
tree_variable_importance = tree_variable_importance,
lecturer_rpart_models = lecturer_rpart_models,
lecturer_rpart_summary = lecturer_rpart_summary,
forest_tuning = forest_tuning,
forest_best_by_type = forest_best_by_type,
forest_best_controls = forest_best_controls,
forest_fit_table = forest_fit_table,
forest_complexity_metrics = forest_complexity_metrics,
forest_min_relative_gain_path = forest_min_relative_gain_path,
selected_forest_fit = selected_forest_fit,
forest_surrogates_by_type = forest_surrogates_by_type,
selected_forest_surrogate = selected_forest_surrogate,
selected_forest_surrogate_summary = selected_forest_surrogate_summary,
forest_selection = forest_selection_table,
forest_variable_importance = forest_variable_importance,
predictive_forest_tuning = ranger_tuning,
predictive_forest_sampling = ranger_sampling_summary,
predictive_forest_feature_lookup = ranger_feature_lookup,
predictive_forest_fit = predictive_forest_fit,
shap_contributions = selected_shap_contributions,
shap_diagnostics = selected_shap_diagnostics,
shap_summary_long = shap_summary_long,
method_comparison = method_comparison_table,
importance_wide = importance_wide
)

save(drc_results_objects, file = results_object_file)

data.table(
  saved_to = results_object_file,
  object_count = length(drc_results_objects),
  object_names = paste(names(drc_results_objects), collapse = ", ")
)

```

```

                                                                    saved_to
                                                                    <char>
1: C:/Users/moses.mburu.FIND/Pictures/personal/thesis-mse/data/drc_report_results_objects.rda
  object_count
    <int>
1:           58

1: generation_params, source_data_file, paper_reference_file, outcome_labels, congo_model_dt, raw_co

```